```
paxEmvBehaviorSplit(-tests) · from the code, 2026-11-02 · before 27/10 =
transactionOwnsSteps (30cd5aa) · last device test: 19cc019 (chip, tap again)
```

مسارات الـ EMV في EasyLink

تشرح هذه الصفحة رحلة العملية من زر البيع حتى الـ 2nd GENERATE AC، ثم تأكيد العملية أو إلغاؤها عند الـ host، للـ contact والـ contactless؛ وكيف تصل الـ params من التنزيل إلى الـ kernel. كل اسم في الصفحة موجود في الكود، وكل سهم يمثل استدعاءً حقيقيًا.

---

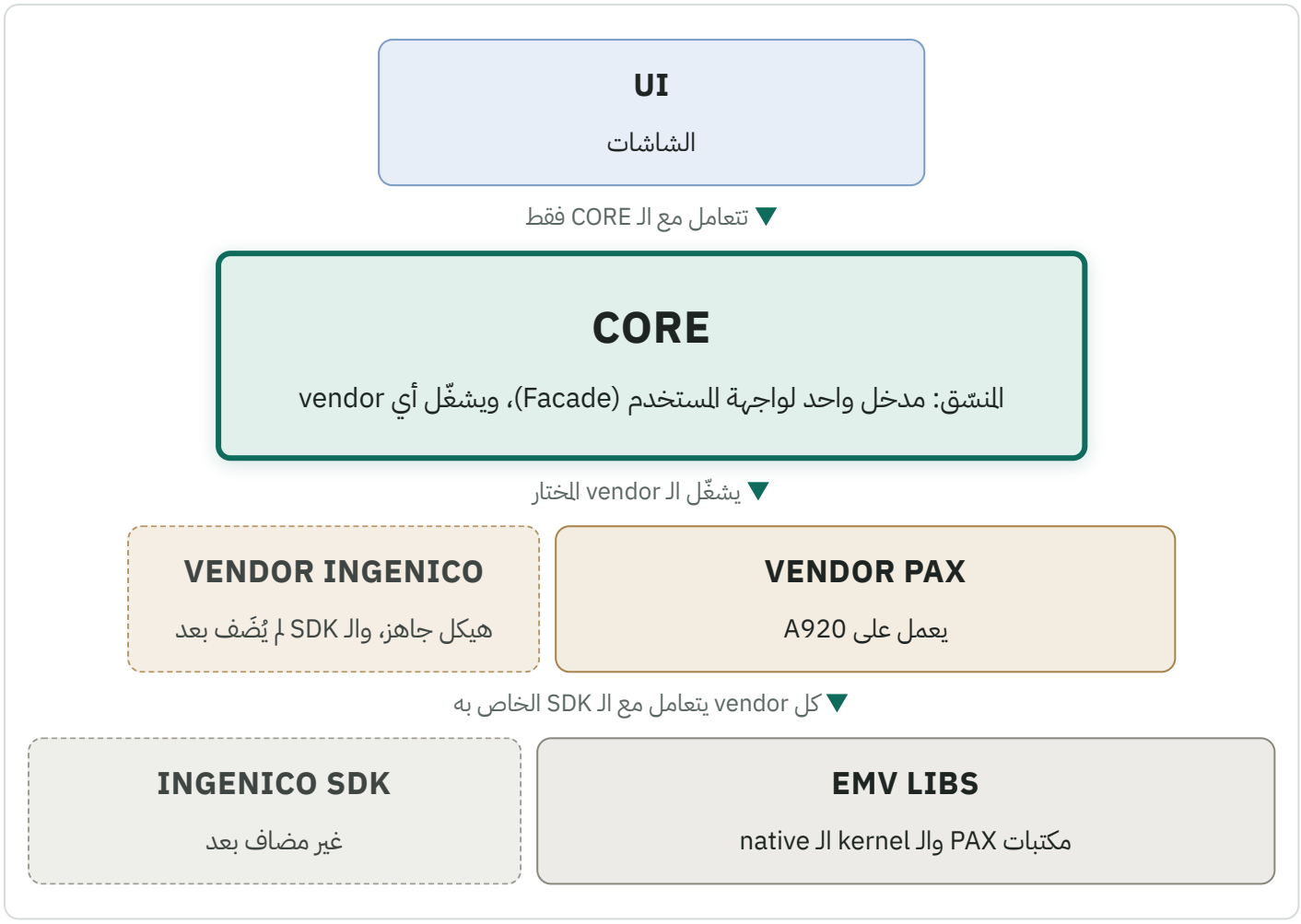
1. المكوّنات ومسؤولية كل منها
2. بداية العملية ونهايتها
3. تشغيل خطوات الـ EMV: من goToStep إلى onXxx
4. حين ينتظر الـ kernel ردًا: askStep
5. الطريق إلى الـ host: البنك كـ callback
6. الطريق إلى البنك من الداخل ومفاهيم OOP
7. لماذا attach، لا Dagger
8. من الـ ARQC حتى نهاية العملية
9. الـ closings سلسلة steps (فرع transactionStepObserver)
10. مسار الـ contact كاملًا
11. النتيجة، ونهاية العملية، والتأكيد أو الإلغاء عند الـ host
12. مسار الـ contactless
13. الـ EMV params
14. مبادئ وأنماط البرمجة

1. المكوّنات ومسؤولية كل منها

لكل مسؤولية مالك واحد. واجهة المستخدم لا تعرف شيئًا عن PAX، والـ engine لا يعرف حالة العملية، والـ behavior لا يتعامل مع الـ threads ولا مع حالة الانشغال.

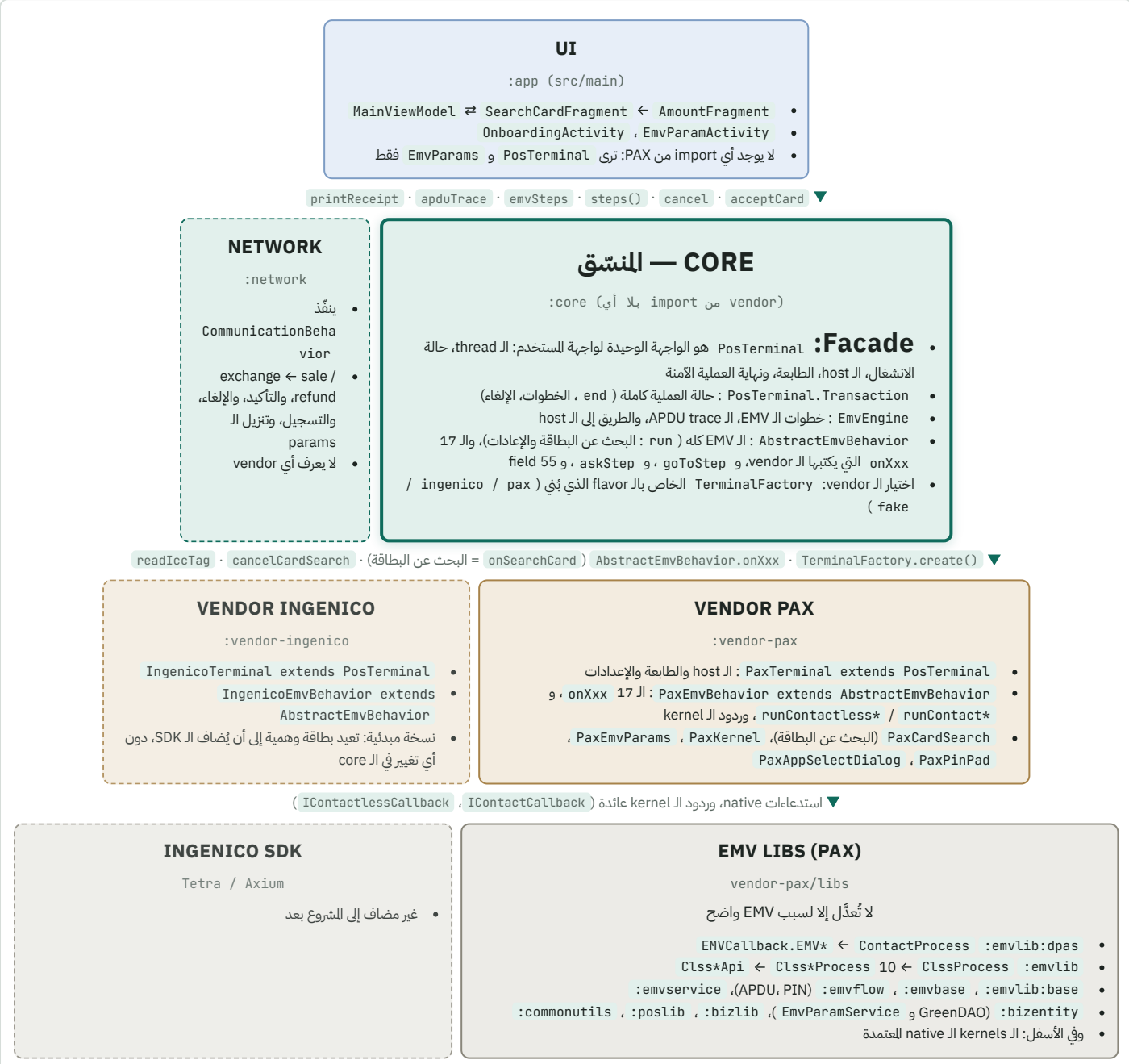
الطبقات (الصورة المبسطة)

يتكوّن الكود من أربع طبقات، وكل طبقة تتعامل مع الطبقة التي تحتها فقط. في المنتصف يقع الـ **CORE**: واجهة المستخدم لا تتعامل إلا معه، وهو الذي يشغّل الـ vendor المختار.


UI
الشاشات
▼ تتعامل مع الـ CORE فقط
CORE
المنسّق: مدخل واحد لواجهة المستخدم (Facade)، ويشغّل أي vendor
▼ يشغّل الـ vendor المختار
VENDOR INGENICO
هيكل جاهز، والـ SDK لم يُضَف بعد
VENDOR PAX
يعمل على A920
▼ كل vendor يتعامل مع الـ SDK الخاص به
INGENICO SDK
غير مضاف بعد
EMV LIBS
مكتبات PAX والـ kernel الـ native

الطبقات بالتفصيل

كل مربع هو module في الكود، وتحته أهم الكلاسات. ما بين الطبقتين هو الواجهة الوحيدة التي تمر من طبقة إلى التي تحتها.



- **الاستدعاءات تتجه إلى الأسفل فقط.** ما يصعد إلى الأعلى هو الخطوات (`Transaction.steps()` ، `emvSteps`)، وردود الـ kernel التي تعود إلى الـ vendor الخاص بها.
- **إضافة vendor جديد** تعني module جديدًا فيه `TerminalFactory` ، وكلاس يرث `PosTerminal` (الإعدادات)، وكلاس يرث `AbstractEmvBehavior` (الخطوات، ومنها البحث عن البطاقة). لا تتغير واجهة المستخدم ولا الـ CORE (`doc/VENDOR.md`).
- يوجد أيضًا `:vendor-fake` للاختبار، ويثبت أن الـ CORE يعمل دون أي SDK.

مكوّنات كل طبقة

كل بطاقة تمثل طبقة من الهرم، وبداخلها الـ packages، وكل كلاس ووظيفته في سطر.

UI `:app (src/main)`

الشاشات. تطلب من الـ CORE وتعرض ما يعود منه، ولا تحتفظ بأي حالة للعملية ولا تعرف أي vendor.

بدء التطبيق

`EmvEnhanceApp`

ينشئ `PosTerminal` واحدًا طوال عمر التطبيق (`TerminalFactory.create`)، ويحدد عنوان الـ host

عملية البيع

`MainActivity`

الشاشة الرئيسية: أزرار فقط

`AmountActivity` / `AmountFragment`

نوع العملية والمبلغ، ثم شاشة البطاقة

`SearchCardFragment`

شاشة العملية: الشريط العلوي، الإلغاء، والنتيجة

`MainViewModel`

يربط `PosTerminal` بالـ LiveData: `acceptCard` ، `cancel` ، الخطوات، الـ trace

`Event`

قيمة تُقرأ مرة واحدة (بسبب تدوير الشاشة)

الإعدادات

`EmvParamActivity` / `EmvParamListFragment`

الـ EMV params: من الـ cache أو من قاعدة البيانات، والتنزيل

`OnboardingActivity`

تسجيل الجهاز عند الـ host، و mToken، و PIN bypass

`CountryListActivity`

الدولة والعملة

CORE — المنسّق `:core`

العملية من بدايتها إلى نهايتها، بالكود نفسه لأي vendor: واجهة المستخدم تتعامل معه وحده، وهو يتعامل مع الـ vendor من خلال واجهات (abstract classes و interfaces).

`terminal` — الجهاز

`PosTerminal`

الـ Facade: المدخل الوحيد لواجهة المستخدم. thread واحد، حالة الانشغال، الـ host (90 ثانية)، الطابعة، ونهاية العملية الآمنة (التأكيد أو الإلغاء قبل أن يصبح الجهاز متاحًا)

`PosTerminal.Transaction`

عملية واحدة: حالتها (`end`)، وخطواتها التي تشغّلها (بشكل متزامن)، والإلغاء، والطلبات المرسلة إلى الـ host

`EmvBehavior`

الواجهة: `run` · `cancelCardSearch` · `dispose`

`AbstractEmvBehavior`

الـ EMV المشترك: `run` (المحاولات والبحث والإعادة: `requestRetry` / `retryCount()`)، `goToStep` ، `askStep` ، field 55، والـ 17 `onXxx` التي يكتبها الـ vendor. موجود هنا لأنه يستدعي `Transaction` باستدعاءات package-private

`TerminalBusyException`

محاولة بدء عملية بينما أخرى قيد التنفيذ

`transaction` — العملية

`TransactionConfig`

النوع، المبلغ، طريقة إدخال البطاقة، field 55، الـ PIN block، `asyncRequestId`

`TransactionType`

sale / refund / cash in

`TransactionStep / Event`

مراحل العملية: البدء، انتظار البطاقة، الاتصال بالـ host، الموافقة، التأكيد أو الإلغاء... والخطوات التي تقود العملية (البدء، الإلغاء، النتيجة، التأكيد / الإلغاء عند الـ host، النهاية) لكل منها دالة

`emv` — الـ EMV

`EmvEngine`

يرسل كل خطوة EMV إلى من يشغّلها وإلى الشريط العلوي، ويحتفظ بالـ APDU trace. لا علاقة له بحالة العملية ولا بإعادة المحاولة

`EmvStep / Event`

الخطوات الـ 17 الخاصة بالـ kernel (EMV Book 3)

`EmvTransactionResult`

ما قُرئ من البطاقة ويحتاجه الـ host

`Field55`

يبني field 55 من الـ tags بالطريقة نفسها لكل vendor

`ApduTrace`

سجل الـ APDU والخطوات (في نسخة الـ debug فقط)

`card` — البطاقة

`CardPresence / CardSearchListener`

البطاقة التي عُثر عليها، وأحداث البحث

`EntryMethod`

chip / tap / swipe / manual

`host` — الواجهات الخارجية

`CommunicationBehavior`

الـ host: `authorize` · `reverse` · `acknowledge`

`AuthResult`

رد الـ host: approved / declined / failed / notSent

`PrinterBehavior`

الطابعة

`HostDefaults`

host وطابعة وهميان للعرض التجريبي

`cardholder` — انتظار حامل البطاقة

`AbstractAppSelectDialog`

اختيار التطبيق على البطاقة: الانتظار والمهلة، ولا يختار الأول تلقائيًا أبدًا

`PinPadBehavior / AbstractPinPadBehavior / PinKey`

شاشة الـ PIN: النقاط والأزرار فقط، والتشفير من مهام الـ SDK

`util · param`

`EmvParams`

واجهة شاشة الـ EMV params

`EmvLog`

السجلات

NETWORK `:network`

التواصل مع الـ host الخاص بنا. ينفّذ `CommunicationBehavior` ولا يعرف أي vendor.

عملية البيع

`SaleCommunicationBehavior`

sale / refund ← exchange، والتأكيد والإلغاء، ويحدد لكل فشل: هل يمكن أن يكون الطلب قد وصل إلى الـ host أم لا

`HostApiConnection` / `HostApiClient`

الـ endpoints (Retrofit) والـ client

`HostHeaders` / `HostSettings` / `HostAppKeys`

الـ headers: الـ token، و mToken (يُعدَّل من الجهاز)، والمفاتيح الثابتة

`PosEntryModeCodes`

051 / 071 / 901 / 011

`tlv` · `crypto`

`RsaPublicKeyEncryptor`

يشفّر مفتاح العملية (TEK) بمفتاح الـ host

`AesEnvelopeCrypto`

يشفّر الطلب والرد ويفك تشفيرهما

`BerTlv` / `IssuerResponseFields`

يقرأ الـ ARPC والـ scripts من رد الـ host

`onboarding` · **params**

`OnboardingClient` / `OnboardingState`

تسجيل الجهاز، والمفاتيح، والـ token وتجديده

`EmvParamDownloadClient`

ينزّل ملف الـ params (البيانات فقط، والقراءة لدى PAX)

`EnvironmentProvider`

عنوان الـ host الخاص بالـ build

`PinBypassSettings`

هل الـ PIN bypass مسموح أم لا

VENDOR PAX `:vendor-pax`

PAX A920: ينفّذ واجهات الـ CORE باستخدام SDK الخاص بـ PAX. هذا هو المكان الوحيد الذي يظهر فيه اسم PAX.

البدء

`TerminalFactory`

المكان الوحيد الذي يُنشئ `PaxTerminal`

`VendorApplication`

الـ Application الخاص بـ PAX (الـ SDK يحتاج `BaseApplication`)

`PaxHardwarePermissions`

صلاحيات قارئات ICC / Mag / PICC

`PaxSetup`

الدولة والعملة، والبيانات الأولية عند أول تشغيل (خرجت من `PaxTerminal`)

البطاقة والـ EMV

`PaxTerminal`

`extends PosTerminal` : الـ host والطابعة الفعليان، والإعدادات

`PaxCardSearch`

البحث عن البطاقة (polling): عمل الخطوة `SEARCH_CARD` (`onSearchCard`)، و `stop()` عند الإلغاء

`PaxEmvBehavior`

`extends AbstractEmvBehavior`: الـ 17 `onXxx` (استدعاء native لكل خطوة)، والرد على أسئلة الـ kernel

`PaxKernelSession`

ما عُرف خلال المحاولة الحالية، ويُنشأ من جديد مع كل محاولة

`PaxKernel`

كائنات الـ kernel: params، contact، contactless، mag

مساعدات الـ EMV

`PaxPinEntry`

الـ PIN: اللوحة، مفتاح الـ online PIN (AES) ولفّه بـ RSA للـ host، وقواعد الـ bypass. مفتاح الـ host العام يأتيه من `PaxTerminal` ، فلا يقرأ الـ EMV حالة الـ host بنفسه

`PaxHostAnswer`

رد الـ host بالشكل الذي يأخذه الـ kernel للـ 2nd GENERATE AC (`toOnlineResult`)

`PaxReaders`

إغلاق القارئات (mag، ICC، PICC) كلٌّ على حدة

`PaxCardData`

بيانات البطاقة كنص: الـ PAN من track 2 أو tag 5A، وإخفاء الـ PAN، والـ tags النصية (دوال ثابتة، وعليها أول اختبارات PAX)

الشاشات والأجهزة

`PaxPinPad`

شاشة الـ PIN

`PaxAppSelectDialog`

اختيار التطبيق

`PaxPrinter`

الطابعة الحرارية

`PaxEmvParams`

بيانات شاشة EMV Params (الـ cache / قاعدة البيانات / التنزيل)

VENDOR INGENICO `:vendor-ingenico`

الواجهات نفسها تمامًا، لكن بنسخة مبدئية: تعيد بطاقة ونتيجة وهميتين إلى أن يُضاف الـ SDK. عند إضافته لن يتغير الـ CORE ولا واجهة المستخدم.

`TerminalFactory` / `VendorApplication`
البدء

`IngenicoTerminal`
`extends PosTerminal` : الـ host والطابعة الوهميان

`IngenicoEmvBehavior`
`extends AbstractEmvBehavior`: بحث وهمي (`onSearchCard` ؛ لاحقًا Tetra / Axium)، وكل خطوة تنتقل إلى التالية

EMV LIBS (PAX)

`vendor-pax/libs/sdk — emvlib · emvbase · emvservice · bizlib · poslib · commonutils · vendor-pax/libs/ours — emvflow · bizentity`

الـ SDK الخاص بـ PAX والـ kernels الـ native المعتمدة، وكلها داخل الـ vendor في `vendor-pax/libs/` : المجلد `sdk/` هو كود PAX نفسه، و `ours/` هو كودنا (`emvflow` و `bizentity`). لا يتعامل معها إلا vendor-pax (مسارات Gradle مثل `:vendor-pax:libs:sdk:poslib` ، وتُكتب هنا مختصرة `:poslib`)، ولا تُعدَّل إلا لسبب EMV واضح.

الـ kernels

`:emvlib:dpas — ContactProcess`
الـ contact kernel: من `EMVAppSelect` إلى `EMVCompleteTrans` ، و `EmvCallBackListener`

`:emvlib — ClssProcess + Clss*Process`
الـ contactless: kernel لكل scheme (PayPass، PayWave، ...)

`:emvlib:base` / `:emvbase`
الكلاسات الأساسية، والـ params، والنتائج، والردود

الجهاز

`:emvflow`
EmvFlowRuntime (DAL)، EmvDeviceImpl (APDU، PIN، trace)

`:emvservice`
PinService، MagCardService

`:poslib` / `:commonutils` / `:bizlib`
أدوات PAX الخاصة بالجهاز

الـ params

`:bizentity`
GreenDAO (قاعدة البيانات)، `EmvParamService` (الـ cache)، قارئات الـ XML، `EmvParamUpdater` ، `ConfigInit` (أول تشغيل)

`PosTerminal` هو الجهاز نفسه

يمثّل `PosTerminal` جهاز نقطة البيع الذي يستخدمه الكاشير، أيًّا كان نوعه (PAX A920، Ingenico...). لكل جهاز الأجزاء نفسها، ولكل جزء field في الكود:

```
// PosTerminal = the POS machine
protected final AbstractEmvBehavior   behavior;       // the card: EMV with the card (se
protected final CommunicationBehavior communication;  // HTTP to the bank / the facilita
protected final PrinterBehavior       printer;        // the printer: the receipt
private   final Scheduler             transactionScheduler; // one sale at a time, one t
```

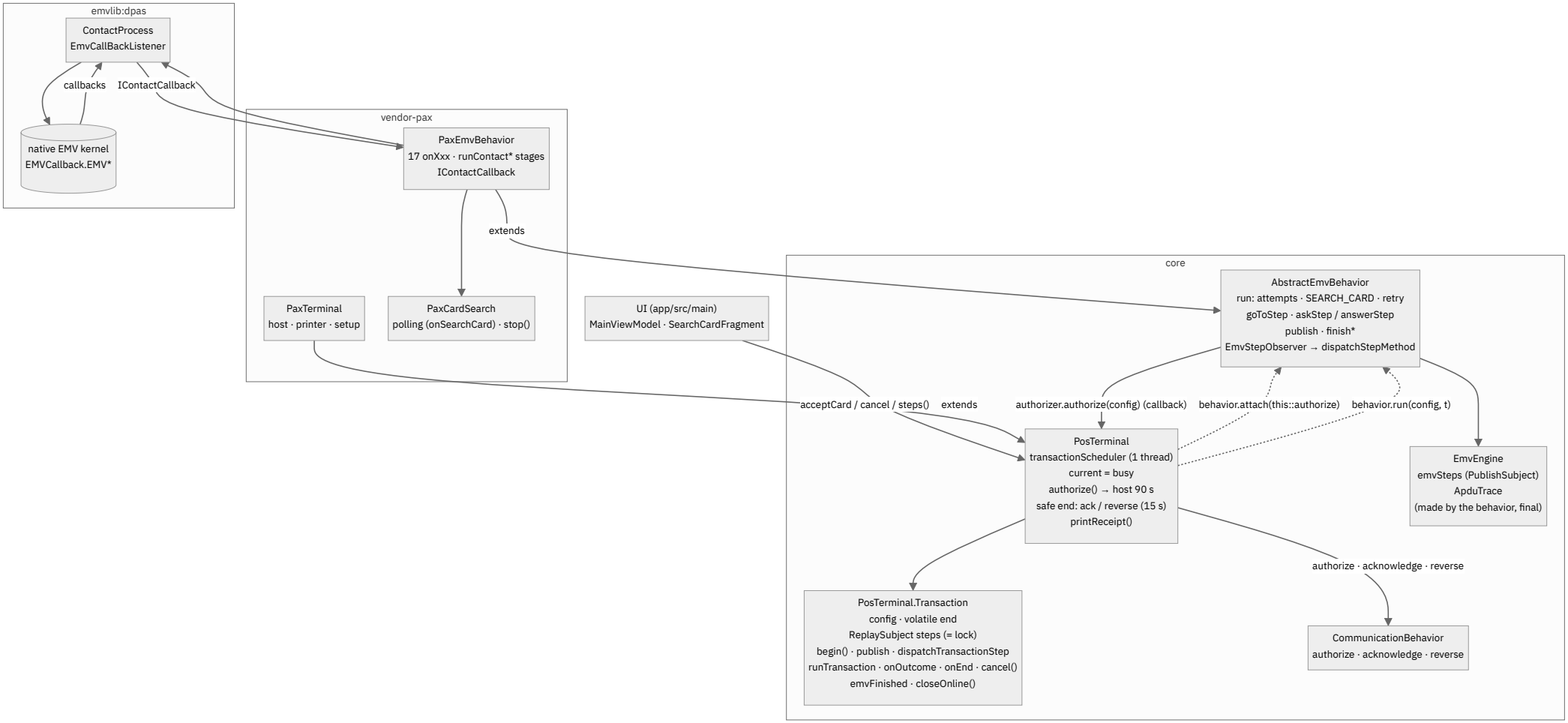
البطاقة ←
`EmvBehavior`
يتعامل مع البطاقة: يبحث عنها، ويقرؤها، وينفّذ خطوات الـ EMV مع الـ kernel، ويجهّز الطلب للبنك. لكل vendor واحد (`PaxEmvBehavior`).

البنك ←
`CommunicationBehavior`
الاتصال بالبنك أو الـ facilitator عبر HTTP: `authorize` (البيع / الاسترداد)، ثم بعد تحديد النتيجة `acknowledge` أو `reverse`. التطبيق الحالي: `SaleCommunicationBehavior`.

الطابعة ←
`PrinterBehavior`
تطبع الإيصال على thread مستقل، بالترتيب. على PAX: `PaxPrinter`.

الجهاز نفسه
ينفّذ عملية واحدة في كل مرة (thread واحد، و `current` تعني أنه مشغول)، ويشغّل الـ EMV (`behavior.run`، والبحث عن البطاقة بداخله)، ويربط هذه الأجزاء معًا.

من يتعامل مع من (الكلاسات)


emvlib:dpas
ContactProcess
EmvCallBackListener
callbacks
IContactCallback
native EMV kernel
EMVCallback.EMV*
vendor-pax
PaxEmvBehavior
17 onXxx · runContact* stages
IContactCallback
extends
PaxTerminal
host · printer · setup
PaxCardSearch
polling (onSearchCard) · stop()
UI (app/src/main)
MainViewModel · SearchCardFragment
core
AbstractEmvBehavior
run: attempts · SEARCH_CARD · retry
goToStep · askStep / answerStep
publish · finish*
EmvStepObserver → dispatchStepMethod
acceptCard / cancel / steps()
extends
authorizer.authorize(config) (callback)
behavior.attach(this::authorize)
behavior.run(config, t)
PosTerminal
transactionScheduler (1 thread)
current = busy
authorize() → host 90 s
safe end: ack / reverse (15 s)
printReceipt()
EmvEngine
emvSteps (PublishSubject)
ApduTrace
(made by the behavior, final)
authorize · acknowledge · reverse
PosTerminal.Transaction
config · volatile end
ReplaySubject steps (= lock)
begin() · publish · dispatchTransactionStep
runTransaction · onOutcome · onEnd · cancel()
emvFinished · closeOnline()
CommunicationBehavior
authorize · acknowledge · reverse

الكلاس	ما يمثله في الواقع	مسؤول عن (وحده)	ممنوع عليه
`PosTerminal`	الجهاز	• عملية واحدة في كل مرة: thread واحد، و `current` تعني مشغول • البنك: `authorize` (بحد أقصى 90 ثانية)، وتسمية كل طلب (`asyncReques` `tId`) • نهاية العملية الآمنة: التأكيد أو الإلغاء قبل أن يصبح متاحًا (بحد أقصى 15 ثانية) • الطابعة	أي منطق EMV، والبحث عن البطاقة
`PosTerminal.Transaction`	العملية الواحدة (بيع / استرداد)	• حالتها: `end` (`null` تعني قيد التنفيذ) • الخطوات التي تراها الشاشة (`ReplayS` `ubject`، وهو أيضًا الـ lock) • الإلغاء • الطلبات المرسلة إلى البنك وردودها	نشر أي شيء بعد انتهائها
`AbstractEmvBehavior`	التعامل مع البطاقة (المشترك بين كل الـ vendors)	• `run` : المحاولات، والبحث عن البطاقة، والإعادة • الـ 17 خطوة (`onXxx`)، و `goToSte` `p` (نقطة فحص الإلغاء الوحيدة)، و `ask` `Step` • 55 field، والطلب إلى البنك (`author` `izeOnline`) • الـ engine (`attach` / `engine()`)	الـ threads، حالة الانشغال، البنك مباشرة
`PaxEmvBehavior`	التعامل مع البطاقة على PAX	• الـ `onXxx` : استدعاء native لكل خطوة • ردود الـ kernel • `onSearchCard` = البحث (`PaxCar` `dSearch`) • `PaxKernelSession` جديد في كل محاولة • حوله كلاسات لكل منها وظيفة واحدة: `PaxPinEntry` (الـ PIN)، `PaxHostA` `nswer` (رد الـ host بشكل PAX)، `PaxR` `eaders` (إغلاق القارئات)، `PaxCardDa` `ta` (بيانات البطاقة كنص)	أن يستدعيه أحد غير الـ dispatch
`EmvEngine`	شريط خطوات الـ EMV	• `emvSteps` (التشغيل والشريط العلوي) • الـ APDU trace • الطريق إلى البنك (`PosTerminal::a` `uthorize`)	أي حالة للعملية
`SaleCommunicationBehavior`	اتصال HTTP بالبنك / الـ facilitator	• sale / refund ← exchange • التأكيد / الإلغاء • تصنيف الفشل: هل وصل الطلب إلى البنك أم لا (`classify`)	أن يقرر التأكيد أو الإلغاء (هذا قرار الجهاز)
`PaxPrinter`	الطابعة	• طباعة الإيصال	—
`ContactProcess`	الغلاف حول الـ kernel	• من `EMVAppSelect` إلى `EMVCompl` `eteTrans` • `EmvCallBackListener` : يجب أن يرد دائمًا بـ `setCallBackResult`	واجهة المستخدم، وانتظار شخص

الواجهة `EmvBehavior` تحتوي على دورة حياة الـ EMV فقط: `run` ، `cancelCardSearch` ، `dispose` . أما `attach` و `engine()` فهما في `AbstractEmvBehavior` فقط، ولهذا فإن نوع `PosTerminal.behavior` هو `AbstractEmvBehavior` . والـ behavior يُنشئ الـ `EmvEngine` بنفسه (حقل `final`)، و `PosTerminal` يعطيه الطريق إلى الـ host فقط: `behavior.attach(this::authorize)` (لأن الـ behavior يُبنى قبل `PosTerminal`).

2. بداية العملية ونهايتها

يجري فحص الانشغال على الـ main thread نفسه (`compareAndSet`)، فتُرفض أي عملية ثانية فورًا بـ `TerminalBusyException` ، دون أي طابور انتظار.

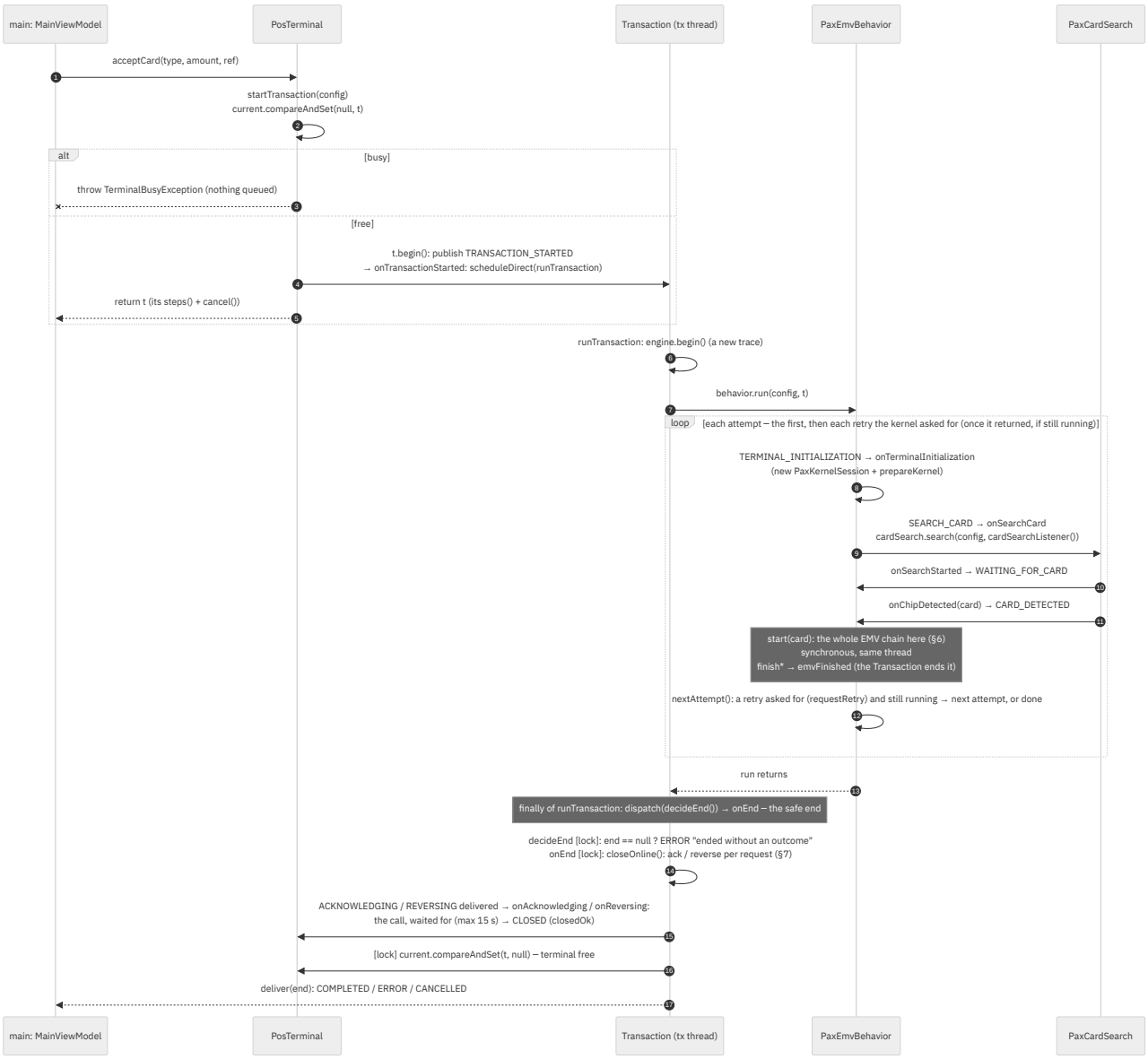
تسير العملية **بخطواتها الخاصة** (TransactionSteps): عند نشر الخطوة تُنفَّذ الدالة الخاصة بها فورًا وعلى الـ thread نفسه، كما هو الحال في خطوات الـ EMV. كل خطوة تقود العملية لها حالة في `dispatchTransactionStep` :

```
switch (event.getStep()) {
    case TRANSACTION_STARTED: onTransactionStarted(event); // to its thread
    case CANCELLING:          onCancelling();              // search stops
    case ACKNOWLEDGING:       onAcknowledging(event);      // host: sale done
    case REVERSING:           onReversing(event);          // host: reverse
    case APPROVED, DECLINED:  onOutcome(event);            // decides the end
    case COMPLETED, ERROR,
         CANCELLED:           onEnd();                     // the safe end
    default:                  // a report (ONLINE_*, CARD_READ…)
}
```

الخطوة العادية تُسلَّم ثم تُشغَّل دالتها. واستثناءان تسلّمهما دالتهما بنفسها: النتيجة (مع النهاية في مسكة واحدة للـ lock، فلا يقع إلغاء بينهما) والنهاية (بعد أن يصبح الجهاز متاحًا). أما `ONLINE_*` فتقارير، لأن `authorize` تعيد رد الـ host مباشرة إلى الـ kernel من أجل الـ 2nd GENERATE AC.

نسخة الـ Rx (الفرع `transactionStepObserver` ، الفرق الوحيد عن هذا الكود): الـ switch نفسه يستدعيه `TransactionStepObserver` مشترك في الـ `steps` ، كما يفعل `EmvStepObserver` في الـ EMV. يبقى الـ lock، وتقرأ `publish` / `end` / `cancel` قيمة `end` قبل دخوله، فلا ينتظر الإلغاء الـ ack. الثمن المقبول: ترى الشاشة `ACKNOWLEDGING` / `REVERSING` بعد انتهاء نداء الـ host لا قبله (BACKLOG 10.4). ولم يعد الـ dispatch هو الفرق الوحيد: في هذا الفرع أيضًا `ENDING` وسلسلة الإغلاق (§7)، والأسماء الجديدة للطريق إلى البنك (§5)، والـ network بخطوات متتالية ومهلة 25 ثانية لكل نداء؛ القائمة كاملة في `doc/REFERENCE.md` §10.

الـ EMV كله داخل `behavior.run` : البحث عن البطاقة (`SEARCH_CARD`) وإعادة المحاولة، كما يفعل الـ Entry Point في EMV Book B. ونهاية العملية **آمنة**: يُرسَل التأكيد أو الإلغاء إلى الـ host ويُنتظر رده قبل أن يصبح الجهاز متاحًا لعملية جديدة. (تعديلات 28/10، لم تُختبر على الجهاز بعد.)



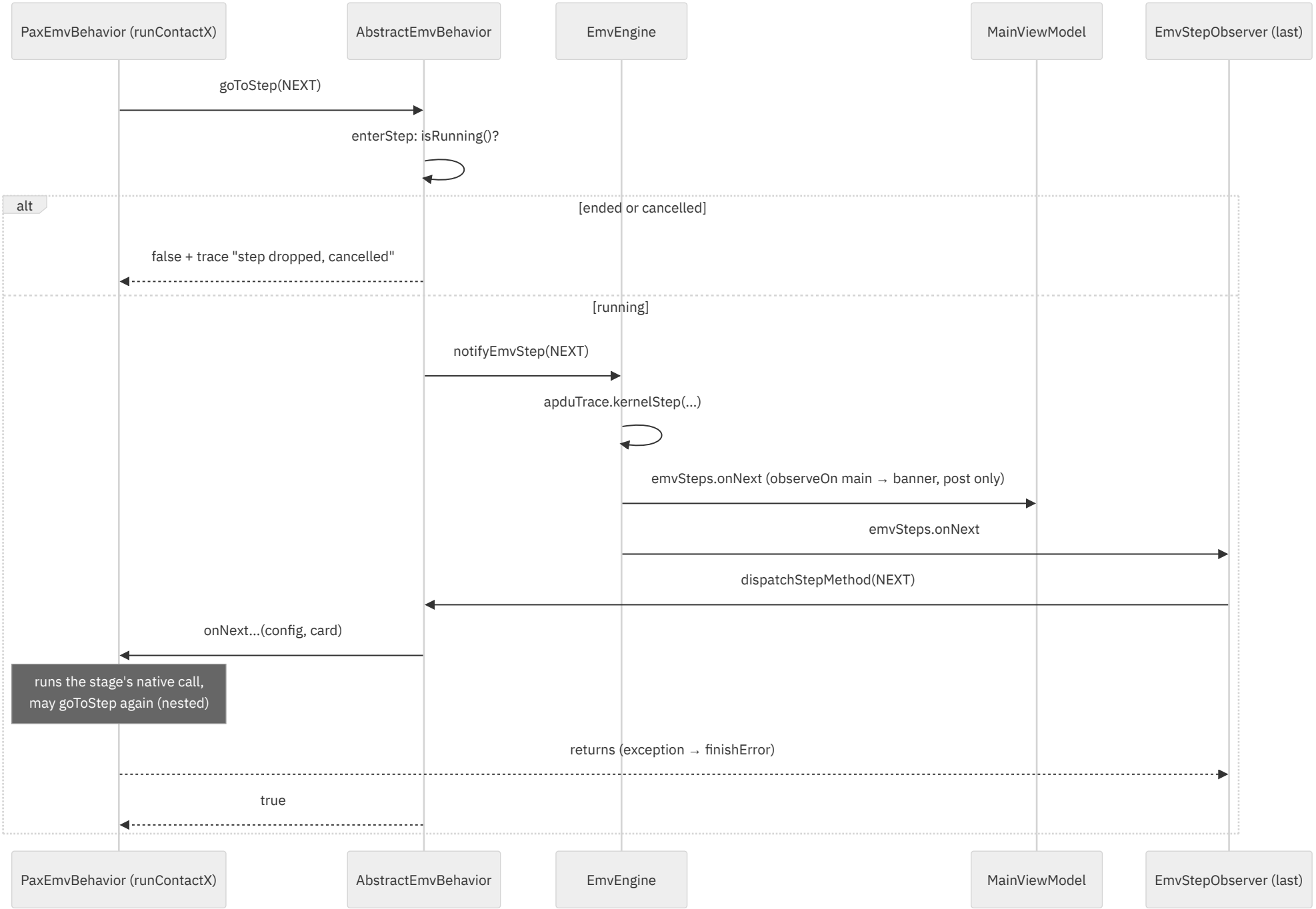
- **الإلغاء** (`Transaction.cancel()` من الـ main thread): داخل الـ lock تُضبط `end = CANCELLED` وتُنشر `CANCELLING` ، ودالتها (`onCancelling`) تستدعي `behavior.cancelCardSearch()` ← `PaxCardSearch.stop()` ← `stopPolling()` . يتوقف البحث، وأي بطاقة يُعثر عليها بعد ذلك تُتجاهل (لأن العملية انتهت)، ويتوقف الـ EMV عند أول `goToStep` . لا يُقاطَع الـ thread أبدًا. ⚠ إلغاء يصل قبل أول `polling()` بلحظة قد يتأخر حتى 30 ثانية (BACKLOG 1.17).
- **إعادة المحاولة من مسؤولية الـ EMV**: يطلبها الـ behavior من داخل الـ kernel عبر `requestRetry(...)` (في `AbstractEmvBehavior` ، فكاتب حالة الإعادة وقارئها هو الـ EMV نفسه) ثم ينهي الـ kernel عمله ويعود. بعدها يأخذ `AbstractEmvBehavior.run` الطلب، فإن كانت العملية ما زالت قائمة (شرط بسيط) يبدأ محاولة جديدة من `SEARCH_CARD` . لا توجد خطوة خاصة بذلك في العملية (أُزيلت `NEXT_ATTEMPT`). العملية نفسها والـ trace نفسه ("RETRY #n")، وأول GENERATE AC بعدها يُسمّى 1st من جديد (`ApduTrace.beginAttempt()`).
- **النهاية الآمنة:** ترى الشاشة `APPROVED → ACKNOWLEDGING → CLOSED → COMPLETED` . محاولة بدء عملية جديدة خلال ذلك تُرفض لأن الجهاز مشغول. إذا لم يرد الـ host على الإلغاء، يُنتظر 15 ثانية ثم تنتهي العملية (دون سجل

محفوظ لإعادة الإرسال: BACKLOG 0.15).

- **الخطوات لواجهة المستخدم:** يعيد `Transaction.steps()` كل الخطوات لأي مشترك جديد (عند تدوير الشاشة)، وكل مشترك محاط بـ `SafeObserver` .

3. تشغيل خطوات الـ EMV: كيف يشغّل `goToStep` الدالة `onXxx`

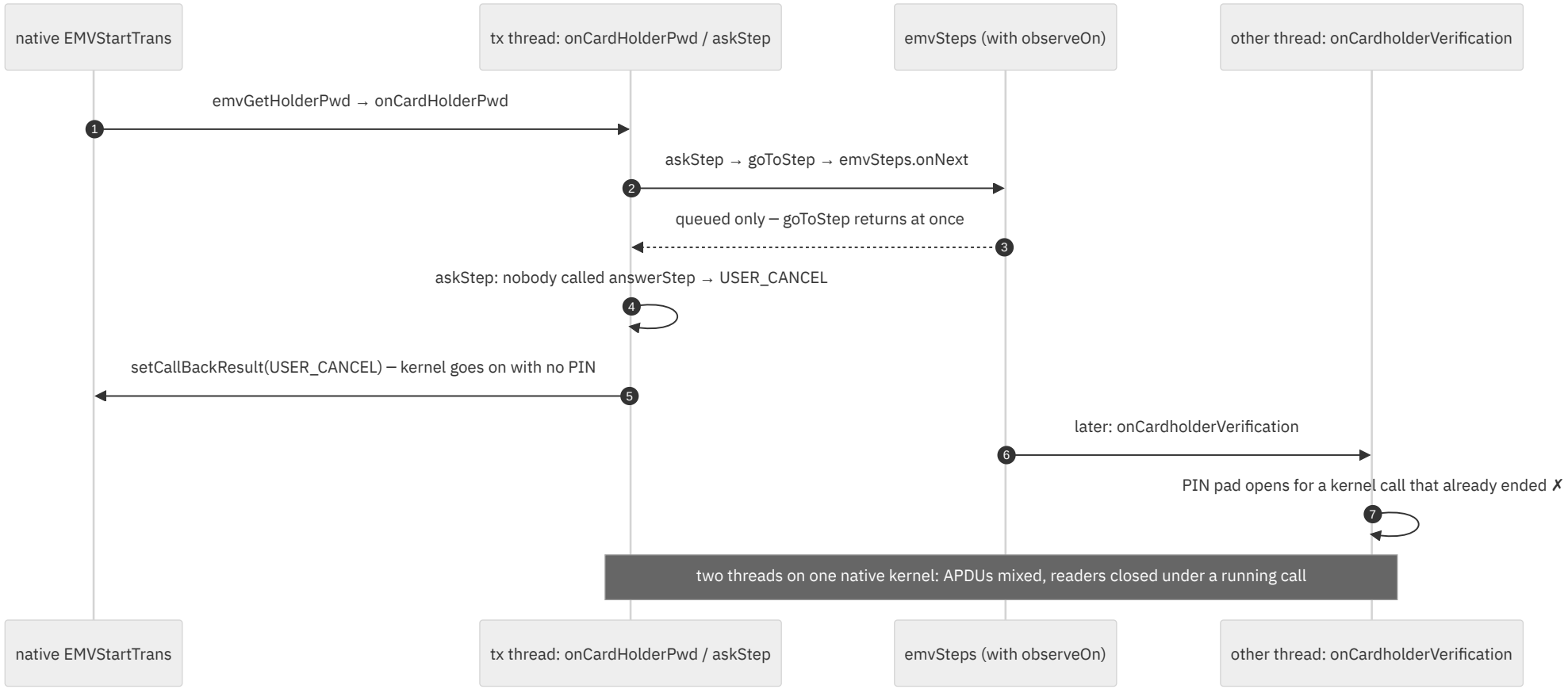
لا يستدعي أحد `onXxx` مباشرة. تُنشر الخطوة على `emvSteps` ، فيشغّل `EmvStepObserver` (آخر مشترك) الدالة الخاصة بها داخل `onNext` ، على الـ thread نفسه. وعندما تستدعي `onXxx` بدورها `goToStep` ، تكتمل الخطوة الجديدة بالكامل قبل أن تعود الأولى (استدعاء متداخل). لذلك يجب أن يبقى هذا التشغيل متزامنًا: بلا `observeOn` وبلا serialize.


PaxEmvBehavior (runContactX)
AbstractEmvBehavior
EmvEngine
MainViewModel
EmvStepObserver (last)
goToStep(NEXT)
enterStep: isRunning()?
alt
[ended or cancelled]
false + trace "step dropped, cancelled"
[running]
notifyEmvStep(NEXT)
apduTrace.kernelStep(...)
emvSteps.onNext (observeOn main → banner, post only)
emvSteps.onNext
dispatchStepMethod(NEXT)
onNext...(config, card)
runs the stage's native call,
may goToStep again (nested)
returns (exception → finishError)
true
PaxEmvBehavior (runContactX)
AbstractEmvBehavior
EmvEngine
MainViewModel
EmvStepObserver (last)

- يشترك `initEmvObserver()` في بداية كل محاولة (`prepare` داخل حلقة `run`)، ليبقى الـ observer **آخر** مشترك. لو لم يكن الأخير، لرأى المشتركون بعده الخطوة التالية قبل الحالية. وإعادة المحاولة محاولة جديدة، فيُعاد الاشتراك.
- تعيد `goToStep` القيمة `false` بعد الإلغاء، فتغلق المرحلة التي تمسك بقارئ الـ ICC القارئ بنفسها (`advanceContactStage` ← `finishContactStage`).

لماذا يجب أن يبقى التشغيل متزامنًا (بلا `observeOn` وبلا serialize)

تعني `observeOn(X)` أن يوضع `onNext` في طابور ويُنفَّذ لاحقًا على thread آخر، وأن تعود `goToStep` فورًا. لو طُبّق ذلك على `EmvStepObserver` لعملت `onXxx` خارج الـ transaction thread، وهذا يكسر ثلاثة أشياء:


native EMVStartTrans
tx thread: onCardHolderPwd / askStep
emvSteps (with observeOn)
other thread: onCardholderVerification
emvGetHolderPwd → onCardHolderPwd
1
askStep → goToStep → emvSteps.onNext
2
queued only — goToStep returns at once
3
askStep: nobody called answerStep → USER_CANCEL
4
setCallBackResult(USER_CANCEL) — kernel goes on with no PIN
5
later: onCardholderVerification
6
PIN pad opens for a kernel call that already ended ✗
7
two threads on one native kernel: APDUs mixed, readers closed under a running call
native EMVStartTrans
tx thread: onCardHolderPwd / askStep
emvSteps (with observeOn)
other thread: onCardholderVerification

لو استخدمنا	ما الذي سيحدث	السبب
`observeOn` على الـ observer	تعيد `askStep` القيمة `USER_CANCEL` قبل أن تُفتح شاشة الـ PIN أو نافذة الاختيار	تعمل الخطوة بعد عودة `goToStep`، فلا يكون أحد قد أجاب في تلك اللحظة
`observeOn` على الـ observer	تداخل على الـ kernel: تستدعي `onXxx` دالة native (`EMVReadAppData`، `EMVCardAuth`) بينما الـ transaction thread ما زال داخل استدعاء native آخر، أو يغلق القارئات في `finishContactStage`	الـ kernel الخاص بـ PAX والقارئ ليسا thread-safe: يجب أن يجري كل شيء على thread واحد وبالترتيب
`observeOn` على الـ observer	يختل الترتيب: يكمل `runContactAppSelect` على افتراض أن مرحلة القراءة انتهت ونتيجتها في `tx.transResult`، وهي لم تنتهِ بعد	كل مرحلة تقرأ نتيجة سابقتها بعد عودة `goToStep`
`emvSteps.toSerialized()`	لا تعمل الخطوة التالية داخل الحالية، بل توضع في طابور حتى ينتهي `onNext` الحالي	كل `onXxx` تستدعي `goToStep(NEXT)` من داخل `onNext`، والـ serialize يؤجّل الاستدعاء المتداخل، فتعيد `askStep` قيمة الأمان وتجد المرحلة النتيجة ناقصة

بدونهما يصبح التشغيل مثل استدعاء دالة عادي: `goToStep` ← `onNext` ← `onXxx`، كله على الـ transaction thread، وينتهي قبل أن تعود `goToStep`. **لا مانع أن تستخدم واجهة المستخدم `observeOn(main)`** (`MainViewModel` على `emvSteps`)، لأنها تعرض الشريط العلوي فقط، ولا تشغّل EMV، ولا ينتظر أحد ردًا منها. هذا الشرط خاص بـ `EmvStepObserver` وحده.

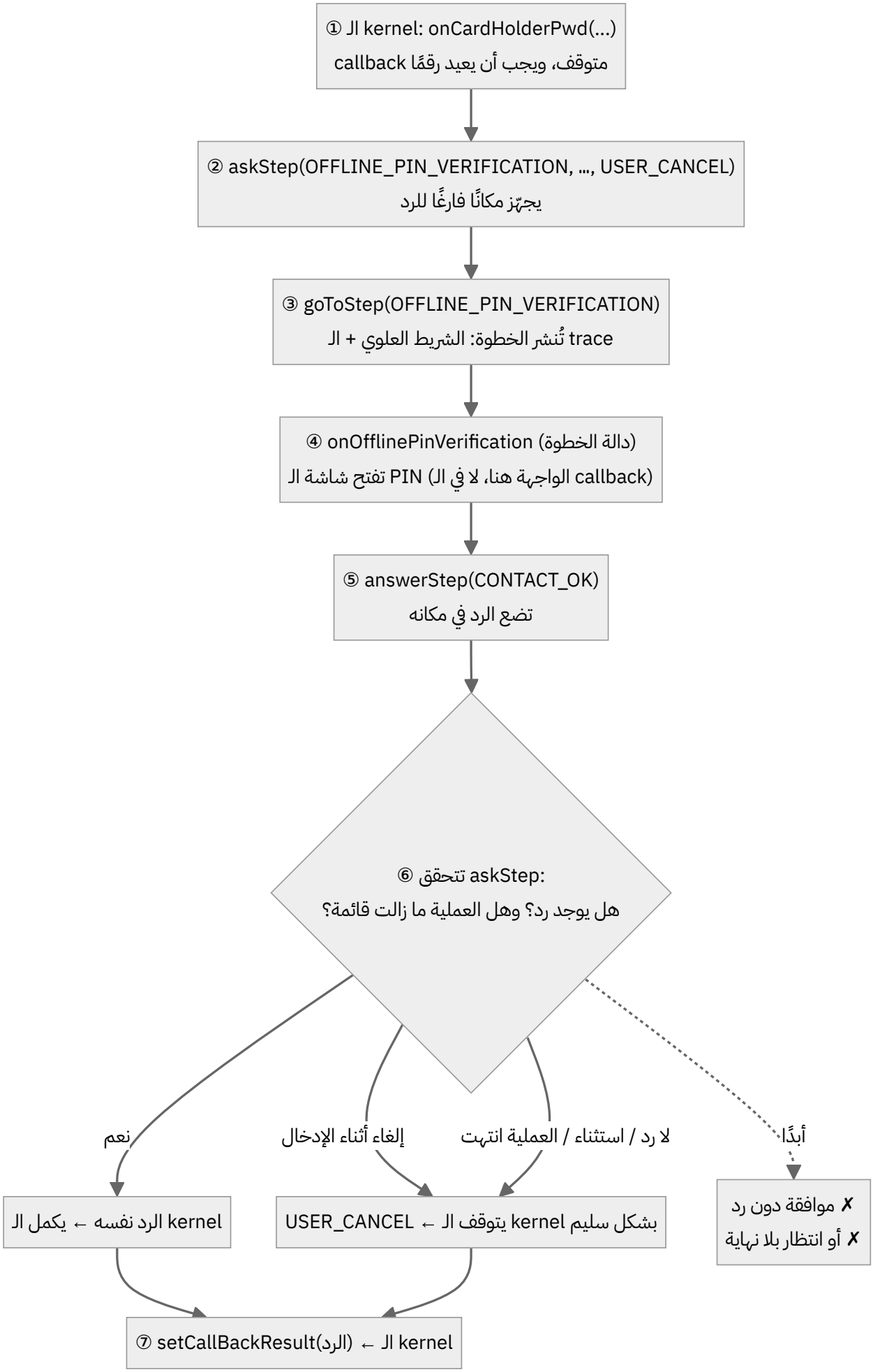
4. حين ينتظر الـ kernel ردًا: الـ callback و `EmvCallBackListener` و `askStep`

أثناء `EMVAppSelect` أو `EMVStartTrans` يتوقف الـ native kernel وينتظر ردًا على callback. يجب أن يعود الرد عبر `setCallBackResult` في كل الحالات، وإلا يبقى الـ kernel منتظرًا. هناك ثلاث طبقات:

الطبقة	أين	ماذا تفعل
Native callback	`ContactProcess.EmvCallBackListener` (implements `EMVCallback.EmvCallbackListener`)	تحوّل البيانات (مثلًا `EMV_APPLIST[]` ← `Candidat`)، وتستدعي الـ listener، وترد بـ `setCallBackResult(EMV_US ER_CANCEL)` إذا رمى الـ listener استثناءً.
محوّل رفيع	`PaxEmvBehavior` بصفته `IContactCallback`: `onWaitAppSelect`، `onCardHolderPwd`، `show ConfirmCard`	لا واجهة مستخدم هنا. يضع الطلب في `tx` ويسأل الخطوة: `askStep(step, detail, USER_CANCEL)`
دالة الخطوة	`onWaitApplicationSelection`، `onCardholderVerification`، `onOfflinePinVerification`	العمل الفعلي (نافذة الاختيار، شاشة الـ PIN)، ثم `answerStep(value)`

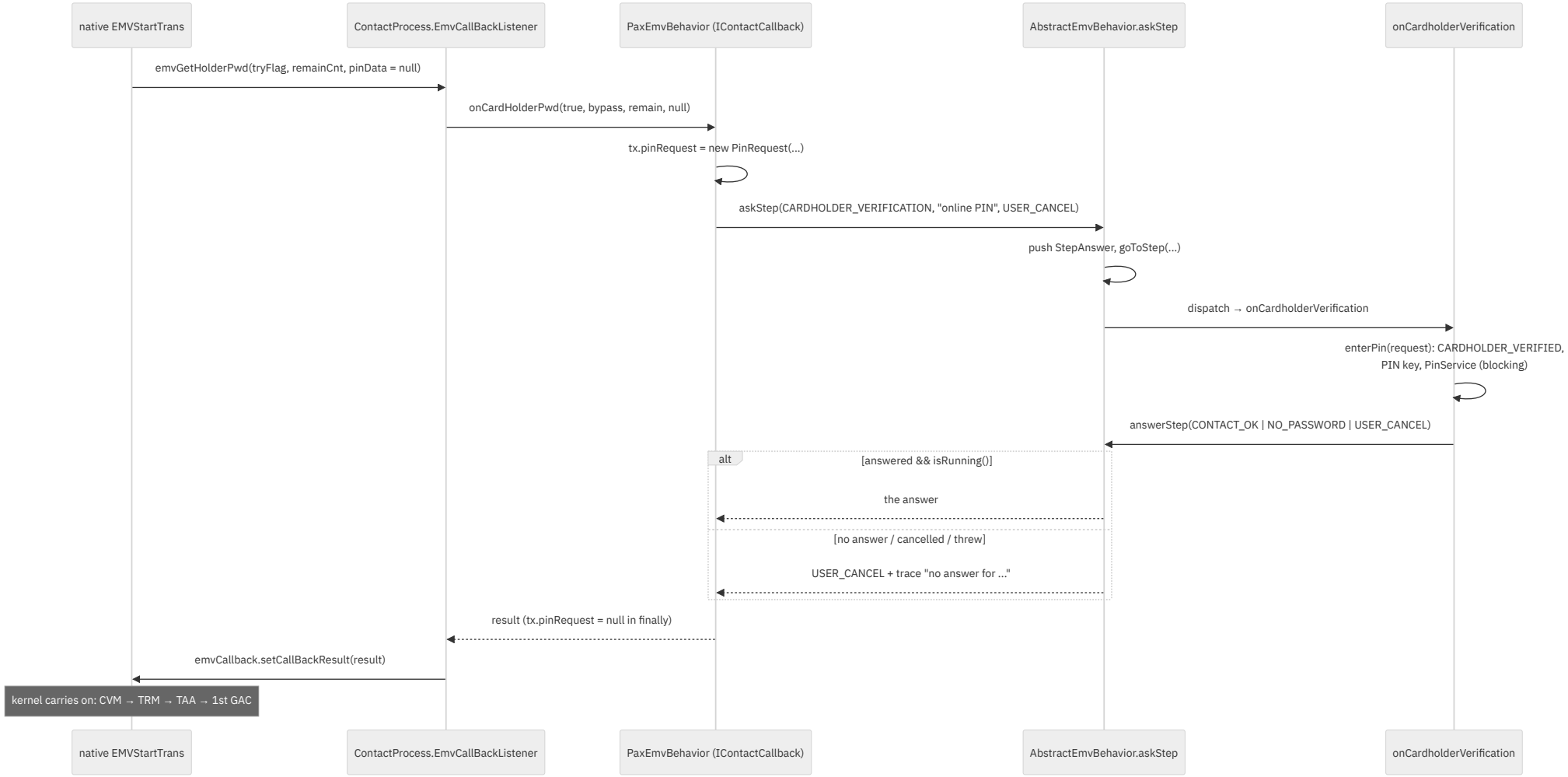
`askStep` في صورة واحدة (offline PIN)

الـ kernel **متوقف بانتظار رقم**. تنقل `askStep` السؤال إلى الخطوة المسؤولة عنه، وتعيد إلى الـ kernel ردًا صحيحًا مهما حدث:


① الـ kernel: onCardHolderPwd(...)
متوقف، ويجب أن يعيد رقمًا callback
② askStep(OFFLINE_PIN_VERIFICATION, …, USER_CANCEL)
يجهّز مكانًا فارغًا للرد
③ goToStep(OFFLINE_PIN_VERIFICATION)
trace تُنشر الخطوة: الشريط العلوي + الـ
④ onOfflinePinVerification (دالة الخطوة)
(callback الواجهة هنا، لا في الـ) PIN تفتح شاشة الـ
⑤ answerStep(CONTACT_OK)
تضع الرد في مكانه
⑥ تتحقق askStep:
هل يوجد رد؟ وهل العملية ما زالت قائمة؟
نعم
إلغاء أثناء الإدخال
لا رد / استثناء / العملية انتهت
أبدًا
kernel الرد نفسه ← يكمل الـ
بشكل سليم kernel يتوقف الـ ← USER_CANCEL
✗ موافقة دون رد
✗ أو انتظار بلا نهاية
⑦ setCallBackResult(الرد) ← الـ kernel

- **لماذا نحتاجها إذا كانت العملية تعمل بدونها؟** في المسار الطبيعي لا فرق. قيمتها حين يُلغي أحد أو لا يرد أحد: هي **نقطة التفتيش** التي تضمن أن يتلقى الـ kernel "إلغاء" فورًا، وتجمع قاعدة الأمان (fail closed، القاعدة 4) في مكان واحد، وتُظهر الخطوة في الشريط العلوي والـ trace. ليست ضرورية لكي تعمل العملية، بل لكي **تتوقف بشكل صحيح**.
- **حدودها:** لا تغلق شاشة الـ PIN نفسها (تبقى إلى أن يُدخل العميل الرقم أو تنتهي مهلتها، ثم يصبح الرد `USER_CANCEL`)، ولا توقف استدعاء native أثناء تنفيذه، فالإيقاف يحدث بين الخطوات فقط.

بالتفصيل (online PIN)


native EMVStartTrans
ContactProcess.EmvCallBackListener
PaxEmvBehavior (IContactCallback)
AbstractEmvBehavior.askStep
onCardholderVerification
emvGetHolderPwd(tryFlag, remainCnt, pinData = null)
onCardHolderPwd(true, bypass, remain, null)
tx.pinRequest = new PinRequest(...)
askStep(CARDHOLDER_VERIFICATION, "online PIN", USER_CANCEL)
push StepAnswer, goToStep(...)
dispatch → onCardholderVerification
enterPin(request): CARDHOLDER_VERIFIED,
PIN key, PinService (blocking)
answerStep(CONTACT_OK | NO_PASSWORD | USER_CANCEL)
alt
[answered && isRunning()]
the answer
[no answer / cancelled / threw]
USER_CANCEL + trace "no answer for ..."
result (tx.pinRequest = null in finally)
emvCallback.setCallBackResult(result)
kernel carries on: CVM → TRM → TAA → 1st GAC
native EMVStartTrans
ContactProcess.EmvCallBackListener
PaxEmvBehavior (IContactCallback)
AbstractEmvBehavior.askStep
onCardholderVerification

- تعمل `askStep` فقط لأن الخطوة تنتهي قبل عودة `goToStep` (§3). واستدعاء `answerStep` دون `askStep` لا يفعل شيئًا.
- الـ offline PIN: تُنشر `CARDHOLDER_VERIFICATION` للإعلان فقط، ثم `askStep(OFFLINE_PIN_VERIFICATION)`. الشاشة فقط، والـ kernel هو الذي يتحقق من الرقم مع البطاقة.
- اختيار التطبيق: `emvWaitAppSel` ← `onWaitAppSelect` ← `askStep(WAIT_ / FINAL_APPLICATION_SELECTION)` ← `answerApplicationSelection` (نافذة اختيار، أو أول تطبيق حين يوجد تطبيق واحد).
- الـ callbacks الخاصة بالبيانات تبقى في `ContactProcess` ولا تسأل أي خطوة: `emvInputAmount`، `emvSetParam` (← `resetParam(AID)`)، `emvUnknowTLVData`. وترد `certVerify` و `emvVerifyPINfailed` بـ `setCallBackResult`.

5. الطريق إلى الـ host: البنك كـ callback (askTheBank ← sendToBank)

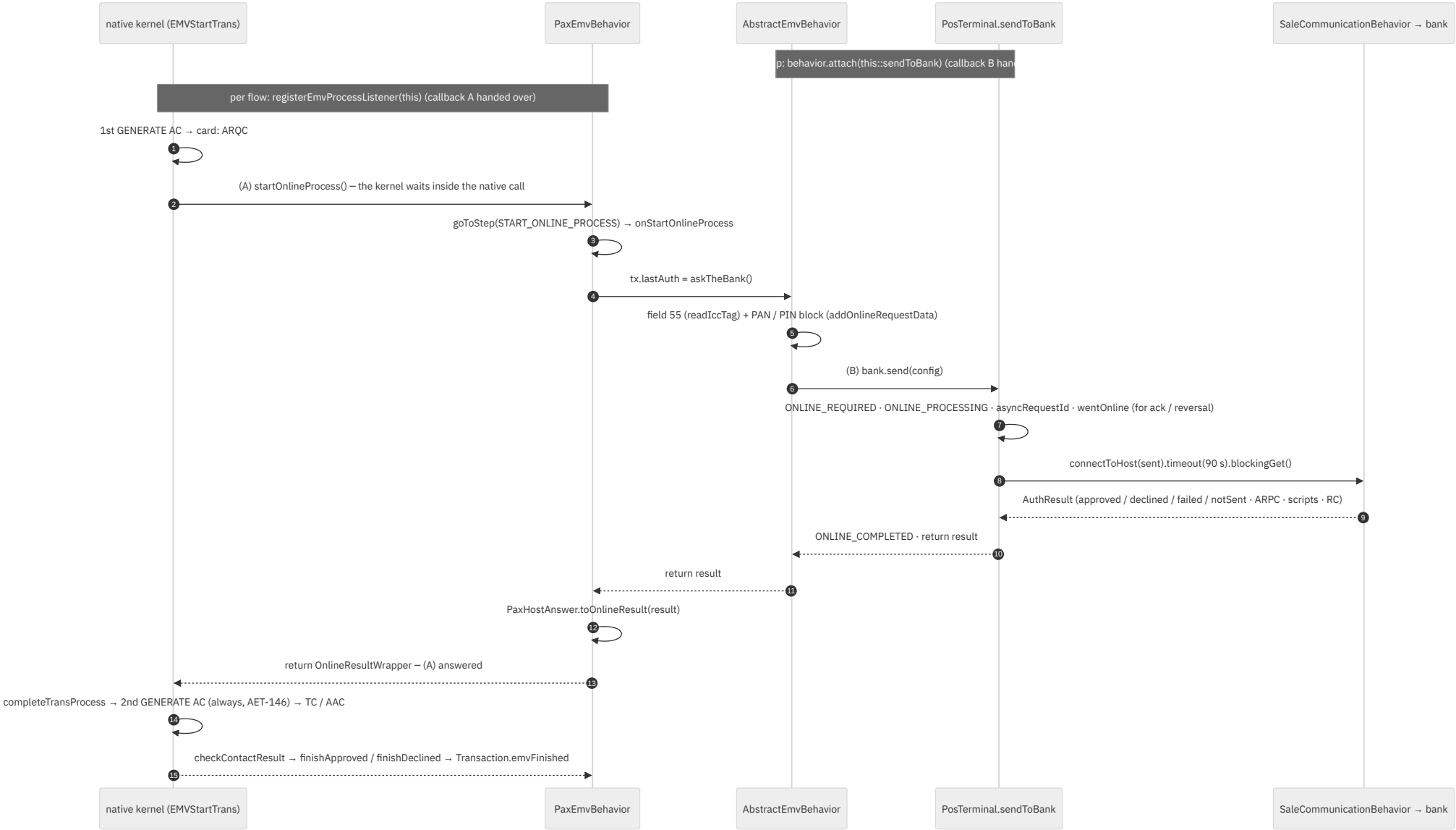
> **الأسماء في هذا القسم** هي أسماء فرع `transactionStepObserver` (`484ed9d` ، `f80691f` ، `5d38ba5`).
> في كود `paxEmvBehaviorSplit` الأسماء الأقدم: `askTheBank` = `authorizeOnline` ،
> `BankRequest.send` = `Authorizer.authorize` (داخل `AbstractEmvBehavior`)،
> `PosTerminal.sendToBank` = `PosTerminal.authorize` ، `connectToHost` =
> `CommunicationBehavior.authorize` . السلوك نفسه.

ما هو الـ callback؟ أن تعطي جهة أخرى "رقمًا تتصل بك عليه"، وهي التي تقرر متى تتصل. أنت لا تستدعي الدالة، بل تسلّمها؛ والطرف الآخر يستدعيها في الوقت الذي يختاره، ثم يكمل بالقيمة التي ترجعها.

في الطريق إلى الـ host **ان-callback**، وكلاهما **متزامن**: من يتصل يتوقف منتظرًا الرد في السطر نفسه.

الـ callback	من سلّم "الرقم"، ومتى	من يتصل، ومتى
أ. `startOnlineProcess()` (`IContactCallbac` `k`)	`PaxEmvBehavior` سجّل نفسه عند الـ kernel: `reg` `isterEmvProcessListener(this)` في بداية الـ contact	**الـ kernel**، بعد الـ 1st GENERATE AC حين يرد الكارت بـ ARQC، وهو واقف داخل الاستدعاء الـ native
ب. `BankRequest`	`PosTerminal` عند إنشائه: `behavior.attach(th` `is::sendToBank)`	`AbstractEmvBehavior.askTheBank` ، بعد تجهيز field 55 وبيانات البطاقة

لا يعرف `AbstractEmvBehavior` شيئًا اسمه `PosTerminal` : معه "رقم" فقط (`bank`). وهذا ما يُبقي طبقة الـ EMV منفصلة عن الـ host.


native kernel (EMVStartTrans)
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal.sendToBank
SaleCommunicationBehavior → bank
p: behavior.attach(this::sendToBank) (callback B han
per flow: registerEmvProcessListener(this) (callback A handed over)
1st GENERATE AC → card: ARQC
1
(A) startOnlineProcess() — the kernel waits inside the native call
2
goToStep(START_ONLINE_PROCESS) → onStartOnlineProcess
3
tx.lastAuth = askTheBank()
4
field 55 (readIccTag) + PAN / PIN block (addOnlineRequestData)
5
(B) bank.send(config)
6
ONLINE_REQUIRED · ONLINE_PROCESSING · asyncRequestId · wentOnline (for ack / reversal)
7
connectToHost(sent).timeout(90 s).blockingGet()
8
AuthResult (approved / declined / failed / notSent · ARPC · scripts · RC)
9
ONLINE_COMPLETED · return result
10
return result
11
PaxHostAnswer.toOnlineResult(result)
12
return OnlineResultWrapper — (A) answered
13
completeTransProcess → 2nd GENERATE AC (always, AET-146) → TC / AAC
14
checkContactResult → finishApproved / finishDeclined → Transaction.emvFinished
15
native kernel (EMVStartTrans)
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal.sendToBank
SaleCommunicationBehavior → bank

الفكرة في سطر: الـ kernel **واقف** داخل الاستدعاء الـ native حتى يرجع (أ)، و(أ) واقف حتى يرجع (ب)، و(ب) واقف حتى يرد البنك أو تنتهي الـ 90 ثانية. كل ذلك على الـ transaction thread وبالترتيب، ولذلك يأخذ الـ 2nd GENERATE AC رد البنك في السطر نفسه. في الـ contactless نستدعي `startOnlineProcess()` نحن من داخل `startContactlessTransProcess`، وقد تكون البطاقة غادرت الحقل، فلا 2nd GENERATE AC (إلا في الـ tap الثاني لـ Visa).

الطريق إلى البنك من الداخل: من أعطى "الرقم" لمن، وبأي مفهوم OOP

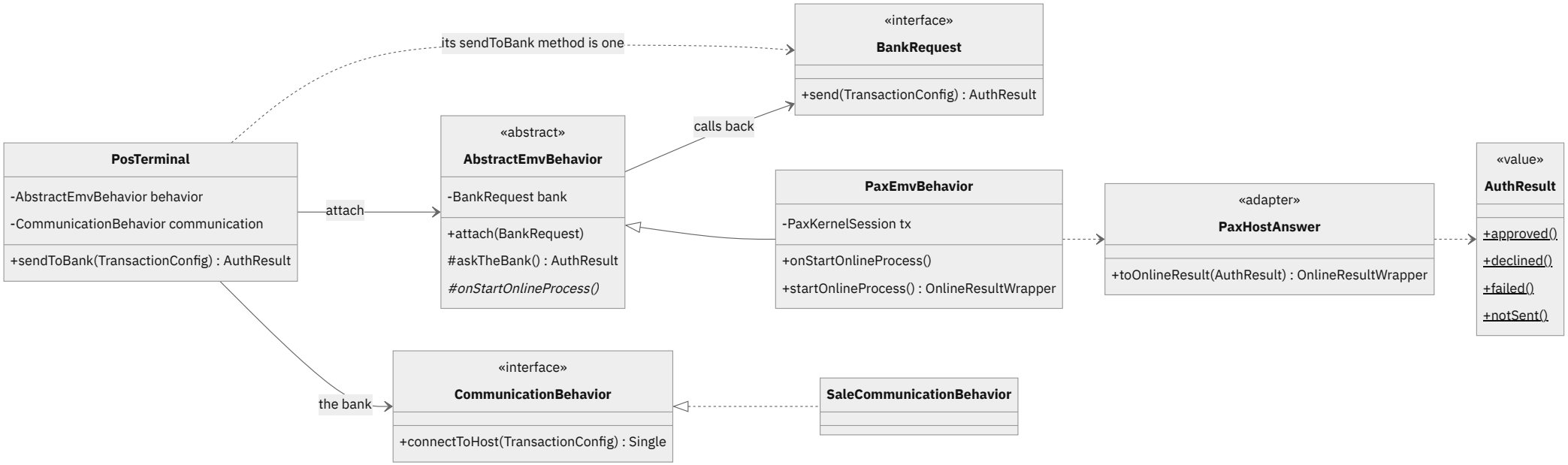
القصة في أربعة أسطر من الكود. `PosTerminal` يعطي الـ behavior "رقمًا يتصل به" مرة واحدة عند إنشائه، والـ behavior يتصل به حين يطلب الـ kernel الذهاب إلى البنك، ثم يرجع الرد إلى الـ kernel في شكل PAX.

```
// ① PosTerminal, once, in its constructor: hand over the number
behavior.attach(this::sendToBank);

// ② BankRequest.java (its own file); AbstractEmvBehavior keeps one — it knows a BankReq
public interface BankRequest { AuthResult send(TransactionConfig config); }
private BankRequest bank;
public final void attach(BankRequest bank) { this.bank = bank; }

// ③ when the kernel asks to go online: call it and wait for the answer
protected final AuthResult askTheBank() {
    ... checks + field 55 ...
    return bank.send(config);                    // = PosTerminal.sendToBank(config)
}

// ④ PAX: keep the answer, then hand it to the kernel in PAX's shape
tx.lastAuth = askTheBank();                   // onStartOnlineProcess
return PaxHostAnswer.toOnlineResult(tx.lastAuth);  // → the kernel
```

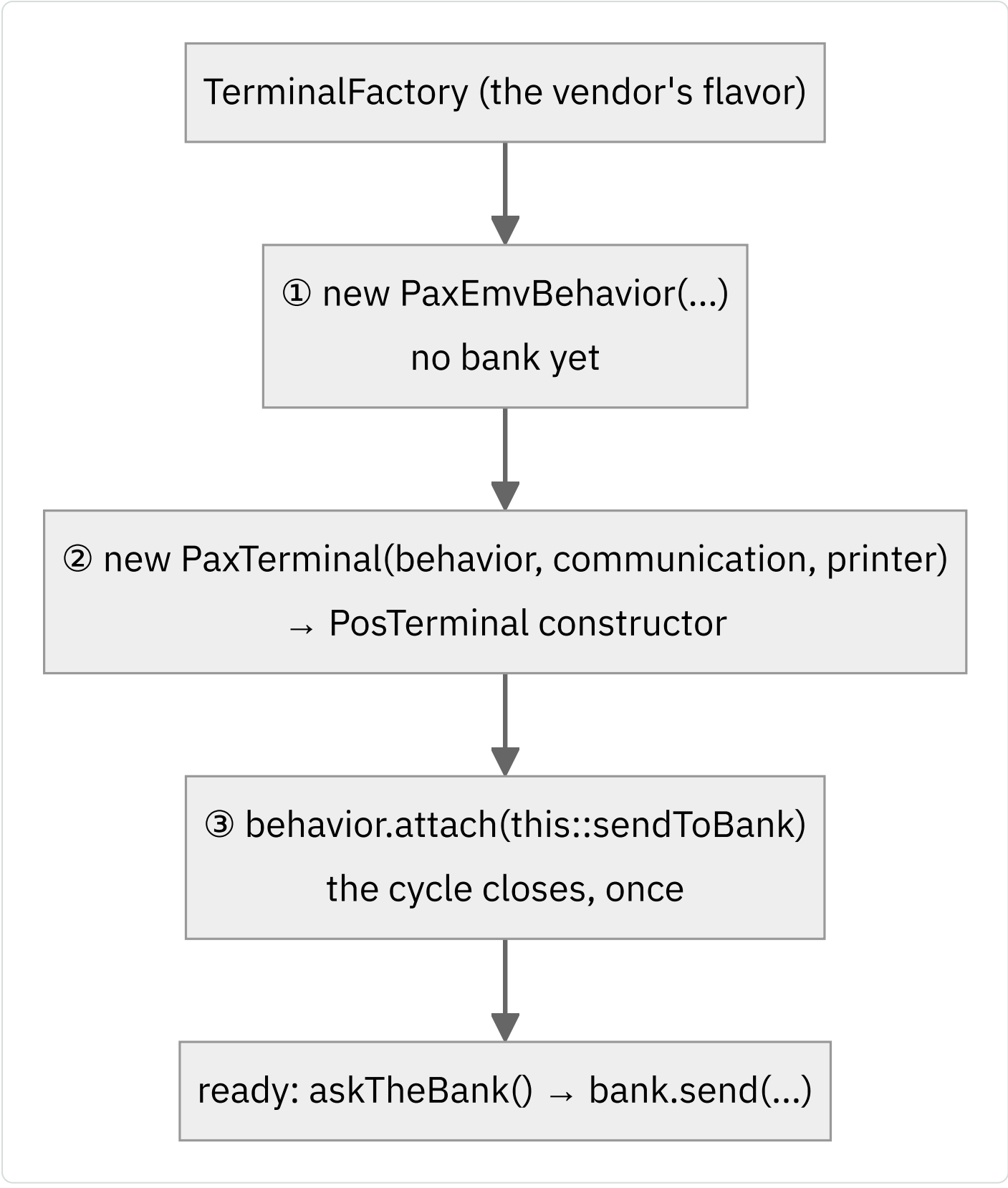

«interface»
BankRequest
+send(TransactionConfig) : AuthResult
its sendToBank method is one
calls back
«abstract»
AbstractEmvBehavior
-BankRequest bank
+attach(BankRequest)
#askTheBank() : AuthResult
#onStartOnlineProcess()
PosTerminal
-AbstractEmvBehavior behavior
-CommunicationBehavior communication
+sendToBank(TransactionConfig) : AuthResult
attach
PaxEmvBehavior
-PaxKernelSession tx
+onStartOnlineProcess()
+startOnlineProcess() : OnlineResultWrapper
«adapter»
PaxHostAnswer
+toOnlineResult(AuthResult) : OnlineResultWrapper
«value»
AuthResult
+approved()
+declined()
+failed()
+notSent()
the bank
«interface»
CommunicationBehavior
+connectToHost(TransactionConfig) : Single
SaleCommunicationBehavior

المفهوم	أين في القصة	ماذا يعطينا
Callback	`attach(this::sendToBank)` ثم `bank.send(config)`	الـ behavior لا يستدعي `PosTerminal` بالاسم؛ معه "رقم" فقط، ويتصل به حين يحتاج.
Dependency Inversion (الـ D في SOLID)	`AbstractEmvBehavior` يعتمد على الـ interface `BankRequest`، لا على `PosTerminal`	طبقة الـ EMV لا تعرف شيئًا عن البنك أو الـ terminal. في الـ tests يُعطى `BankRequest` مزيف بسطر واحد.
Functional interface + method reference	`BankRequest` له method واحدة، و`this::sendToBank` تكتب تنفيذها	لا حاجة لكلاس يطبّق `BankRequest`؛ الـ method الموجودة في `PosTerminal` هي التنفيذ.
Template Method	`askTheBank()` (`final`) في الـ abstract) و `onStartOnlineProcess()` (abstract، كل vendor يملؤها)	الخطوات المشتركة (الفحوص، 55 field، الاتصال) في مكان واحد؛ الـ vendor يقرّر متى يطلبها وماذا يفعل بالرد.
Encapsulation	`private BankRequest bankRequest` و `final attach`	لا يغيّر أحد "الرقم" ولا يتجاوز الفحوص في `askTheBank`.
Polymorphism	`CommunicationBehavior` (التطبيق: `SaleCommunicationBehavior`)، و `AbstractEmvBehavior` (PAX، Ingenico، Fake)	`PosTerminal` يتعامل مع الواجهة؛ يتغيّر البنك أو الـ vendor دون أن يتغيّر هو.
Value Object	`AuthResult` : constructor خاص، وأربع factory methods (`approved` / `declined` / `failed` / `notSent`)، ولا setters	رد البنك شيء ثابت لا يتغيّر بعد إنشائه، وبشكل واحد لا يخص vendor.
Adapter	`PaxHostAnswer.toOnlineResult(AuthResult)`	يحوّل الرد العام إلى شكل PAX (`OnlineResultWrapper` / `IssuerRspData`) الذي يفهمه الـ kernel. vendor آخر يكتب Adapter خاصًا به.

كل ذلك متزامن على الـ transaction thread: الـ kernel ينتظر داخل callback (أ)، و `askTheBank` تنتظر `PosTerminal.sendToBank`، حتى يرجع `AuthResult`.

لماذا `attach`، لا constructor ولا Dagger

المشكلة دائرة. `PosTerminal` يحتاج الـ behavior في الـ constructor الخاص به، والـ behavior يحتاج `PosTerminal::sendToBank`، وهذه لا توجد إلا بعد أن يُنشأ `PosTerminal`. أيهما يُنشأ أولًا لا يجد الآخر.

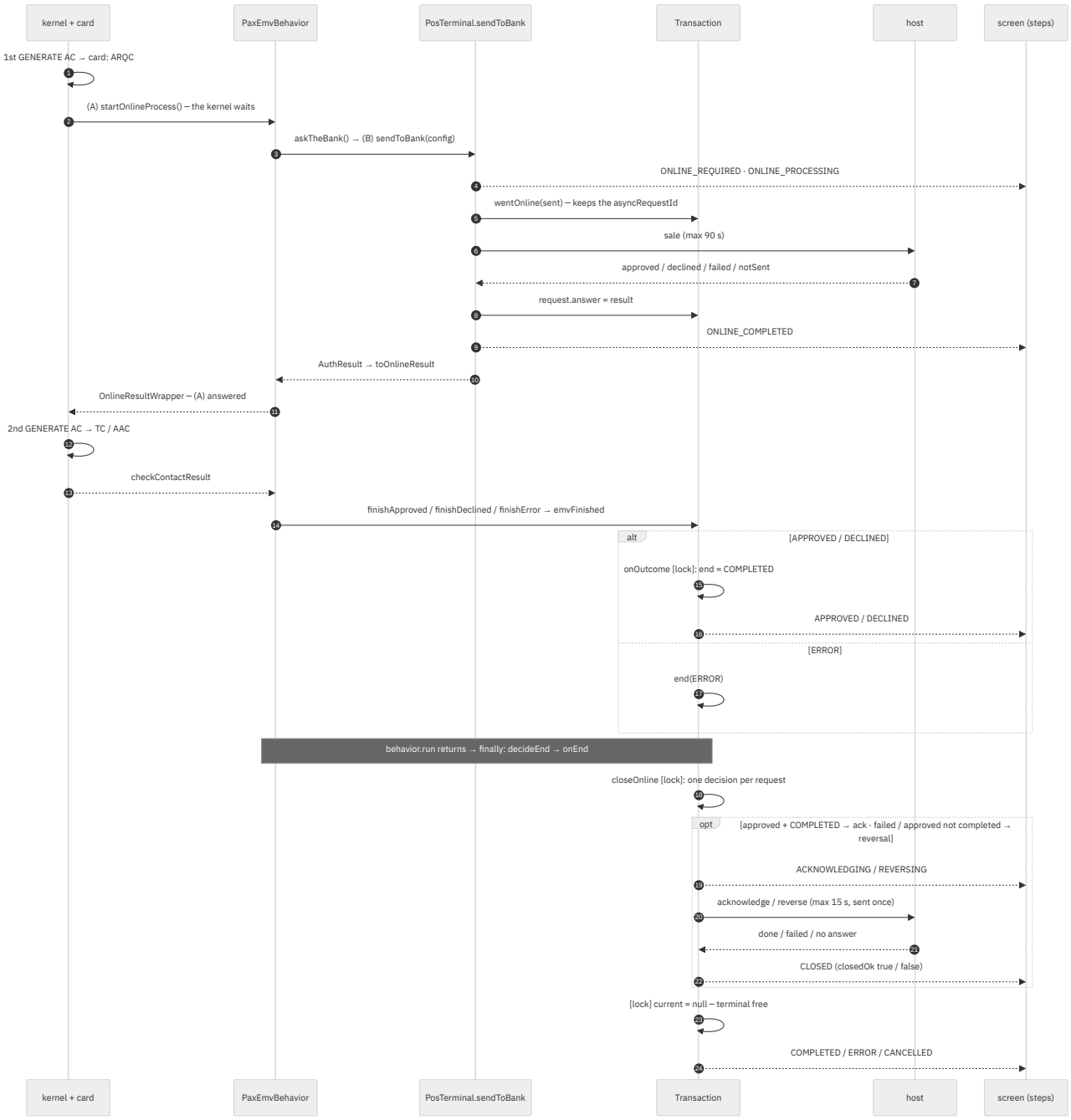

TerminalFactory (the vendor's flavor)
① new PaxEmvBehavior(...)
no bank yet
② new PaxTerminal(behavior, communication, printer)
→ PosTerminal constructor
③ behavior.attach(this::sendToBank)
the cycle closes, once
ready: askTheBank() → bank.send(...)

الحل	ماذا يحدث	القرار
constructor للـ behavior يأخذ `BankRequest`	عند إنشاء الـ behavior لا يوجد `PosTerminal` بعد، فلا يوجد `this::sendToBank` نمرّره	غير ممكن
`PosTerminal` هو الذي يُنشئ الـ behavior	الـ core يصبح مضطرًا لمعرفة كلاسات الـ vendor (`PaxEm vBehavior`)، وهذا يكسر فصل الـ vendor	مرفوض
Dagger / Hilt	يواجه الدائرة نفسها، وحلّه `Lazy<>` أو `Provider<>` أو الحقن بعد الإنشاء: أي `attach` نفسها خلف annotations، مع Modules و Components وكود مولّد ووقت build أطول، لـ graph صغير	لا يبسّط شيئًا هنا
`attach` مرة واحدة في constructor الـ `PosTermin al`	"حقن بعد الإنشاء" (setter injection) واحد: الـ field `private` ، و `final` `attach` ، ويحدث قبل أي عملية	**المختار**

نحن نستخدم Dependency Injection فعلًا، بلا framework (manual constructor injection):
`TerminalFactory` في كل flavor يختار الـ vendor، و `PaxTerminal` يمرّر الأجزاء الثلاثة إلى `PosTerminal` : `super(new PaxEmvBehavior(kernel, pinEntry()), communication, printer)`. لا يُنشئ أي كلاس ما يحتاجه بنفسه، بل يأخذه من الخارج. Dagger / Hilt يصبح مفيدًا حين يكبر الـ graph (شاشات و ViewModels كثيرة تشترك في الـ services)، لكنه لن يلغي `attach` .

من الـ ARQC حتى نهاية العملية (صورة واحدة)

المسار كله على الـ transaction thread، وكل سهم ينتظر الذي قبله. العمود الأخير هو ما تراه الشاشة من خطوات، بالترتيب.

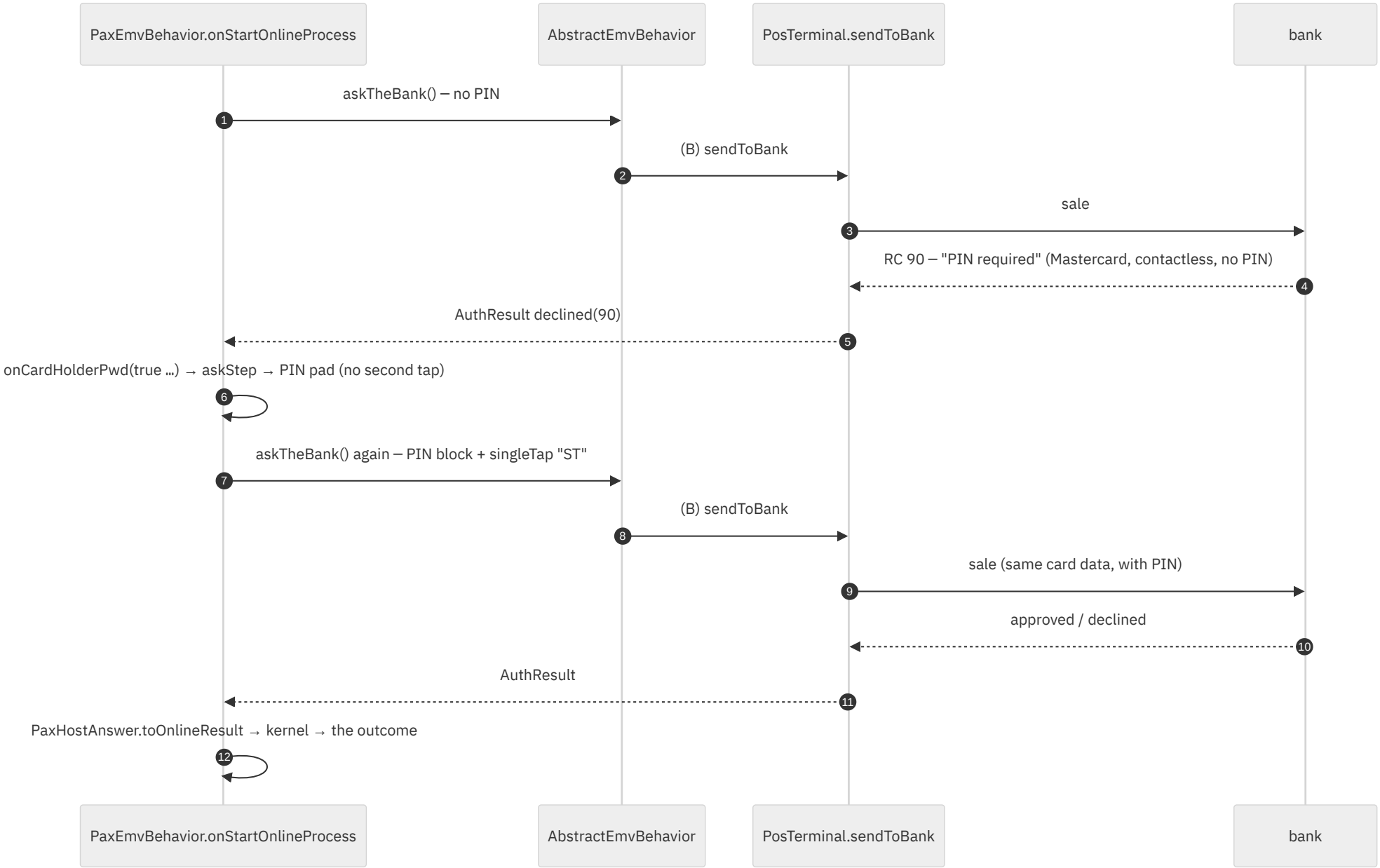


الحالة	ما تراه الشاشة بعد `ONLINE_COMPLETED`
الـ host وافق والبطاقة أعطت TC	`APPROVED → ACKNOWLEDGING → CLOSED → COMPLETED`
الـ host وافق والبطاقة رفضت (AAC)	`DECLINED → REVERSING → CLOSED → COMPLETED`
الـ host رفض	`DECLINED → COMPLETED`
لا رد / انتهاء المهلة (`failed`)	`REVERSING → CLOSED → ERROR`
لم يُرسل (`notSent`)	`ERROR`

⚠ الترتيب مأخوذ من الكود والاختبارات، ولم يُختبر على الجهاز. في فرع `transactionStepObserver` تظهر `ACKNOWLEDGING` / `REVERSING` بعد انتهاء نداء الـ host لا قبله (BACKLOG 10.4).

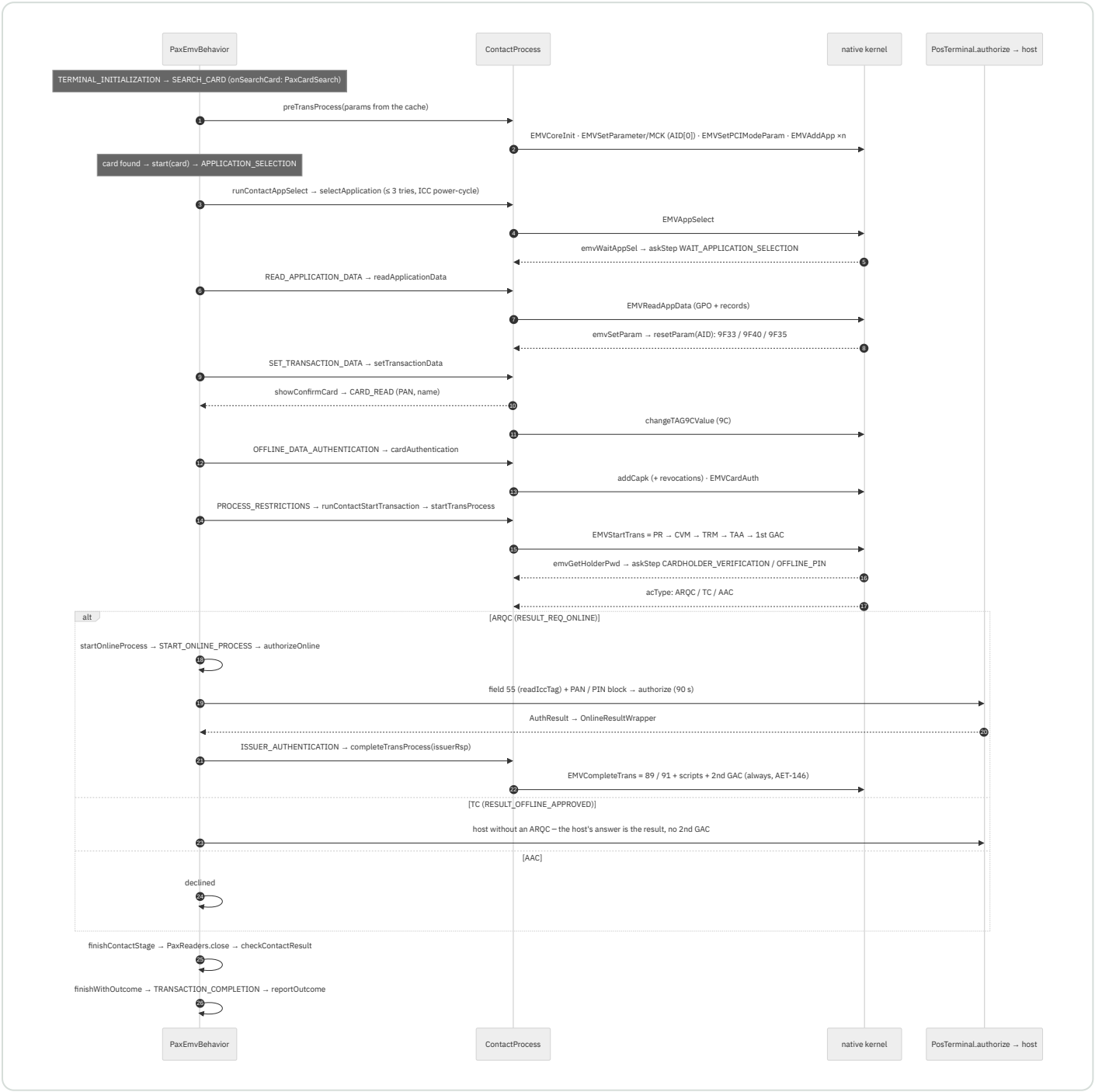
Mastercard Single Tap: البنك يطلب الـ PIN بعد الـ tap

بطاقة Mastercard contactless تُرسل دون PIN (تحت حد الـ CVM الخاص بها)، فيرد البنك بأنه يريد PIN. التطبيق القديم (`SaleActivity`) كان يعرف ذلك من **RC = 90** مع: لا PIN، و contactless، و Mastercard؛ فيفتح لوحة الـ PIN دون tap ثانٍ، ويعيد إرسال البيع ومعه الـ PIN block و `singleTap = "ST"` . التنفيذ عندنا قيد العمل (BACKLOG 4.11):


PaxEmvBehavior.onStartOnlineProcess
AbstractEmvBehavior
PosTerminal.sendToBank
bank
askTheBank() – no PIN
1
(B) sendToBank
2
sale
3
RC 90 – "PIN required" (Mastercard, contactless, no PIN)
4
AuthResult declined(90)
5
onCardHolderPwd(true …) → askStep → PIN pad (no second tap)
6
askTheBank() again – PIN block + singleTap "ST"
7
(B) sendToBank
8
sale (same card data, with PIN)
9
approved / declined
10
AuthResult
11
PaxHostAnswer.toOnlineResult → kernel → the outcome
12
PaxEmvBehavior.onStartOnlineProcess
AbstractEmvBehavior
PosTerminal.sendToBank
bank

6. مسار الـ contact كاملًا (بترتيب الـ kernel نفسه)

كل مرحلة = استدعاء native واحد = `EmvStep` واحدة. وكل دوال `runContact*` بالشكل نفسه: تستدعي الـ kernel، فإن كانت `ret == EMV_OK` ولا توجد نتيجة نهائية تنتقل بـ `advanceContactStage(NEXT)`، وإلا تستدعي `finishContactStage()` التي تحرر الـ listener الخاص بالـ kernel، وتغلق القارئات، وتحسب النتيجة. يُسجَّل الـ listener مرة واحدة في بداية المسار (`runContactAppSelect`).



متى يذهب الطلب إلى الـ host: كما في demo الـ PAX، حين يرد الكارت بـ ARQC (`RESULT_REQ_ONLINE`) ثم يأتي الـ 2nd GENERATE AC. والاسترداد يطلب ARQC كالبيع (جُرّب على بطاقات production في 01/11). الاستثناء الوحيد: TC (`RESULT_OFFLINE_APPROVED`) على جهاز online فقط، فيُرسل إلى الـ host دون ARQC ويكون رده هو النتيجة. أما AAC فرفض، في البيع والاسترداد.

EmvStep	الدالة في `PaxEmvBehavior`	`ContactProcess`	native	الـ callbacks بداخلها
`TERMINAL_INITIALIZATION`	`runPreTransProcess` → `prepareKernel`	`preTransProcess`	`EMVCoreInit`, `EMVAddAp`	—
`SEARCH_CARD`	`PaxCardSearch.search` → `onSearchCard`	—	DAL polling	—
`APPLICATION_SELECTION`	`runContactAppSelect`	`selectApplication`	`EMVAppSelect`	→ `emvWaitAppSelAskStep`
`READ_APPLICATION_DATA`	`runContactReadAppData`	`readApplicationData`	`EMVReadAppData`	`emvSetParam`
`SET_TRANSACTION_DATA`	`runContactSetTransactionData`	`setTransactionData`	(9C) —	`showConfirmCard`
`OFFLINE_DATA_AUTHENTICATION`	`runContactCardAuth`	`cardAuthentication`	`EMVCardAu`, `EMVAddCAPKth`	—
`PROCESS_RESTRICTIONS`	`runContactStartTransaction`	`startTransProcess`	`EMVStartTrans` (PR+CVM+TRM+TAA+1st GAC)	→ `emvGetHolderPwdAskStep`
`START_ONLINE_PROCESS`	→ `startOnlineProcess` `onStartOnlineProcess`	—	(host) —	—
`ISSUER_AUTHENTICATION`	`runIssuerAuthentication`	`completeTransProcess`	`EMVCompleteTrans` (2nd GAC)	—
`SCRIPT_PROCESSING`	إعلان من `completeApproved`	—	(تمت داخل EMVCompleteTrans)	—
`TRANSACTION_COMPLETION`	`r` → `finishWithOutcome` `eportOutcome`	—	—	—

> ⚠ **ما زال كذلك:** ليس للـ 1st GAC خطوة مستقلة (هو داخل `EMVStartTrans`)، و `TERMINAL_RISK_MANAGEMENT` / `TERMINAL_ACTION_ANALYSIS` لا تُنشران أبدًا (لا يوجد استدعاء native منفصل لهما). (يصل الآن إعداد الـ PIN bypass الخاص بالـ AID المختار: `resetParam` يستدعي `EMVSetMCKParam`، وقد أُصلح BACKLOG 0.7.)

7. النتيجة ونهاية العملية

الـ host	البطاقة في 2nd GAC	`TransResultEnum`	النهاية	مع النهاية
APPROVE	TC	`ONLINE_APPROVED`	APPROVED → COMPLETED	acknowledge
APPROVE	غير ذلك	`ONLINE_CARD_DENIED`	DECLINED → COMPLETED	reverse
FAILED (لا رد / انتهاء المهلة / لم يُرسل)	أي نتيجة (الجهاز يطلب AAC)	`ONLINE_FAILED` / `FAILED_CARD_APPROVED`	ERROR "Online failed: <<step>>..."	reverse، إلا إذا كانت `notSent`
DENIAL	—	`ONLINE_DENIED`	DECLINED → COMPLETED	لا شيء

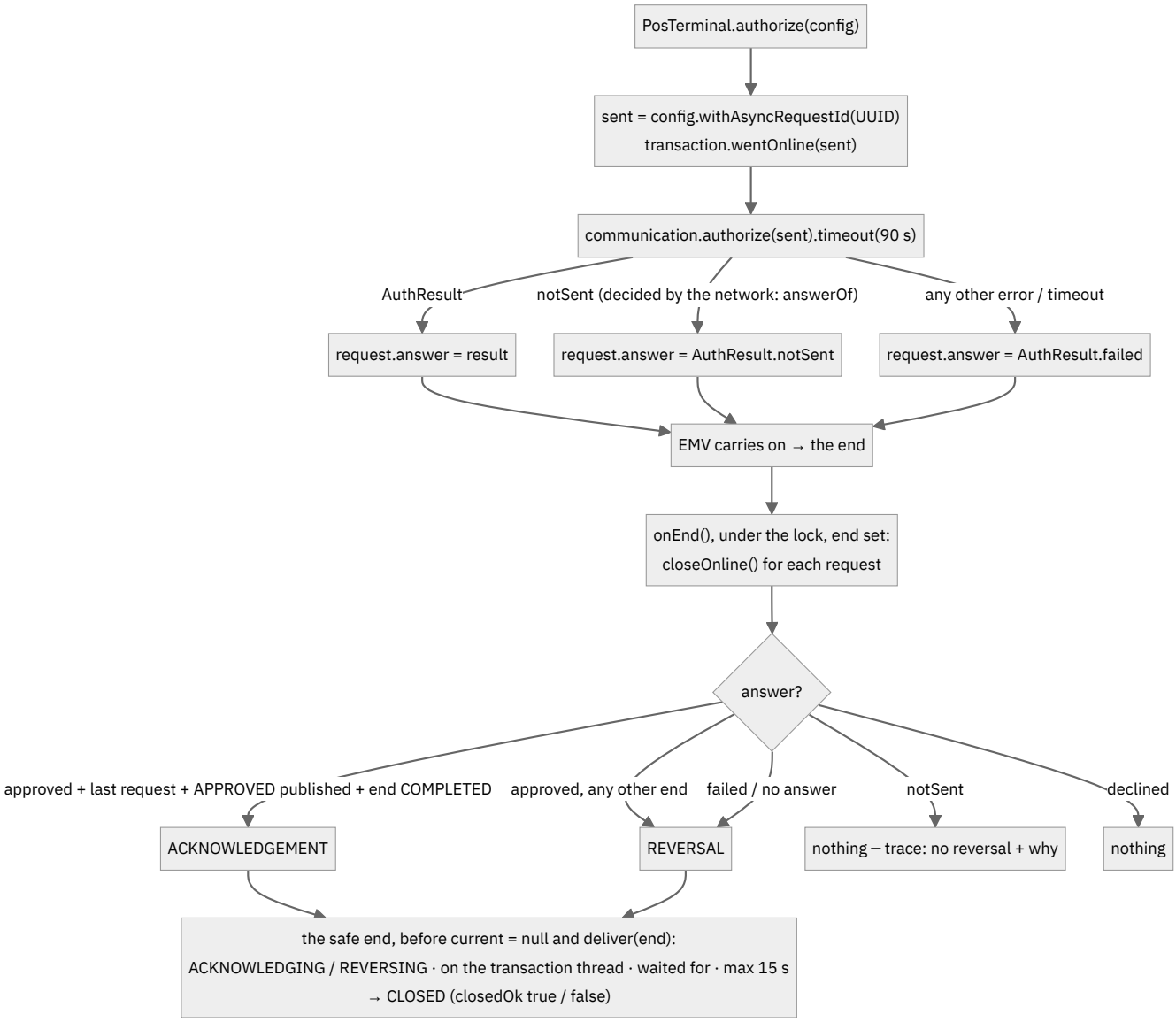
- تصل النتيجة إلى واجهة المستخدم على مراحل: أولًا `APPROVED` / `DECLINED` (وفيها PAN، TVR، TAC، IAC، field 55) فور تحديدها؛ ثم `ACKNOWLEDGING` / `REVERSING` إن وُجد طلب للـ host (النهاية الآمنة)، وبعد كل منهما `CLOSED` بنتيجته (`closedOk`)؛ ثم `COMPLETED`، وهي الـ `end` وتُنشر في `onEnd` (الـ `finally` في `runTransaction`) بعد أن يصبح الجهاز متاحًا.
- `OFFLINE_APPROVED` تُعامل كـ online failed (الجهاز online-only)، و `FALLBACK` تعني إعادة المحاولة بالشريط المغناطيسي.

> ⚠ **مخاطر مفتوحة:** الإلغاء أثناء انتظار رد الـ host ينهي العملية بـ CANCELLED حتى لو وافق الـ host وأعطت البطاقة TC، وفي هذه الحالة يُرسل إلغاء (reversal) لذلك الطلب. يُرسل الإلغاء مرة واحدة: إذا فشل يُسجَّل في الـ log والـ trace فقط، دون سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).

الـ EMV يبلّغ، والـ `Transaction` تُنهي العملية (`28d79cb`): ترسل `finishApproved` / `finishDeclined` / `finishError` النتيجة إلى `Transaction.emvFinished`، التي تنشرها: APPROVED / DECLINED تشغّل `onOutcome` التي تضبط النتيجة والنهاية معًا داخل الـ lock، و ERROR تصبح النهاية كما هي. إلغاء يسبقها يجعل الشاشة ترى CANCELLED فقط (لم يعد ممكنًا أن تظهر APPROVED ثم CANCELLED). وهذا هو المكان الذي سيُتخذ فيه قرار Mastercard single tap.

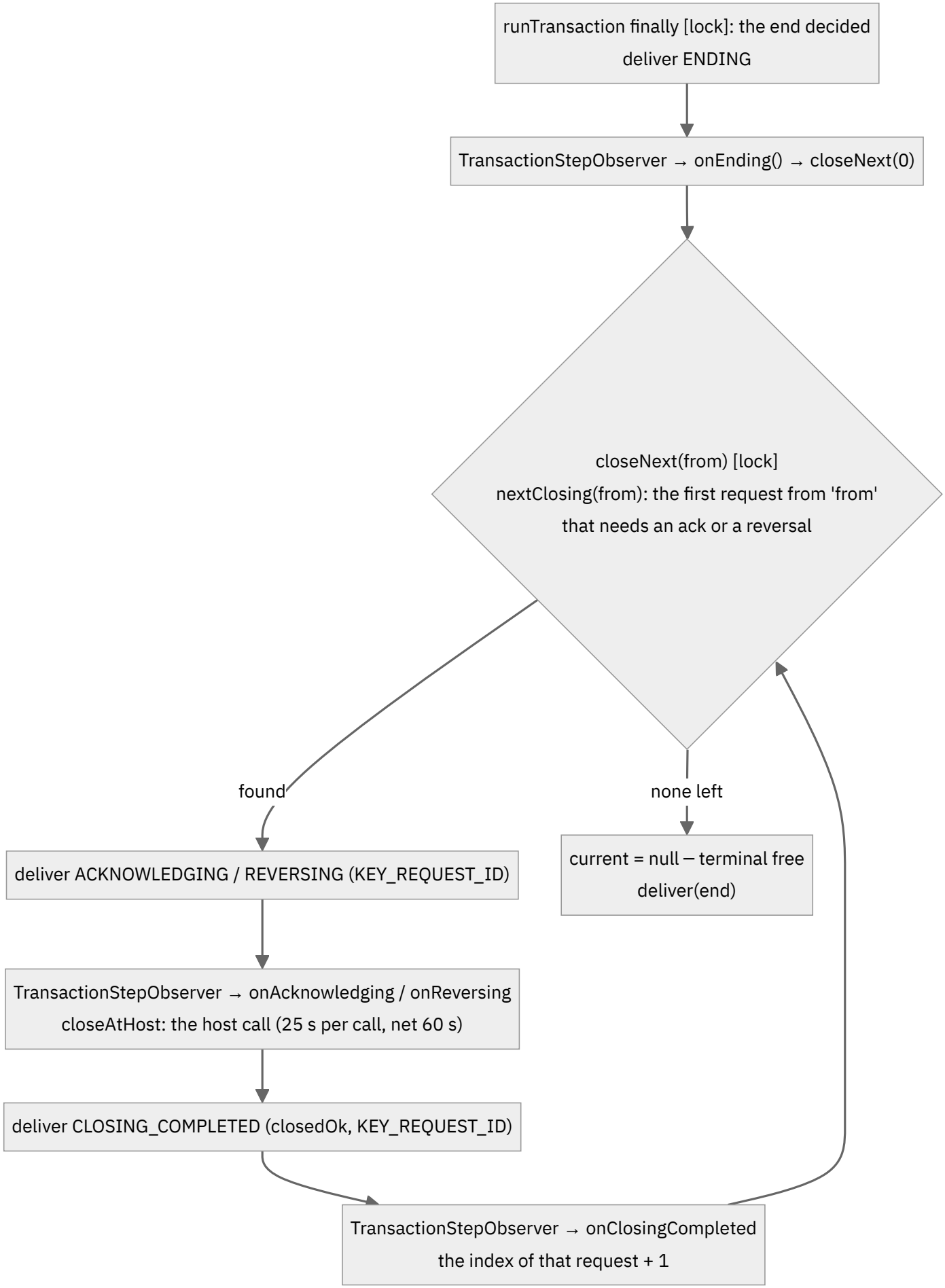
النهاية الآمنة: التأكيد (acknowledgement) أو الإلغاء (reversal)

يُغلَق كل طلب أُرسل إلى الـ host عند تحديد النهاية، و**قبل** أن يصبح الجهاز متاحًا (28/10، `9278c35`): إما **acknowledgement** (العملية اكتملت) أو **reversal** (يلغيها الـ host). الواجهة هي نفسها في التطبيق القديم: `orchestration/acknowledgement` و `orchestration/reverse`، والـ body هو `{asyncRequestId, transactionType}`، بصيغة JSON عادية وبـ headers البيع نفسها. القرار لـ `PosTerminal`، والـ network وحده يحدد ما إذا كان الفشل قد يكون وصل إلى الـ host: يرد بـ `AuthResult` جاهز (`notSent` أو `failed`)، عبر `answerOf`، ولا يبقى لـ `PosTerminal.failureOf` إلا مهلة الـ 90 ثانية الخاصة به.



فرع `transactionStepObserver` : النهاية والـ closings سلسلة steps

في هذا الكود تبني `onEnd` قائمة الـ closings كلها وترسلها في loop، ثم تحرّر الجهاز وترسل النهاية. في فرع `transactionStepObserver` لا يوجد loop (`c2ec7f2` ، `d5cbe8f` ، `6899b84`): الـ `finally` يسلّم `ENDING` ، ودالتها تبدأ السلسلة، وكل `CLOSING_COMPLETED` يحرّك ما بعده، حتى النهاية. وكل الخطوات يدخلها الـ observer وحده.



- `closeOnline()` التي كانت ترجع قائمة أصبحت `nextClosing(from)` : ترجع أول طلب يحتاج closing أو `null` . القرار لكل طلب (ack أو reversal أو لا شيء) وملاحظات الـ trace كما هي.

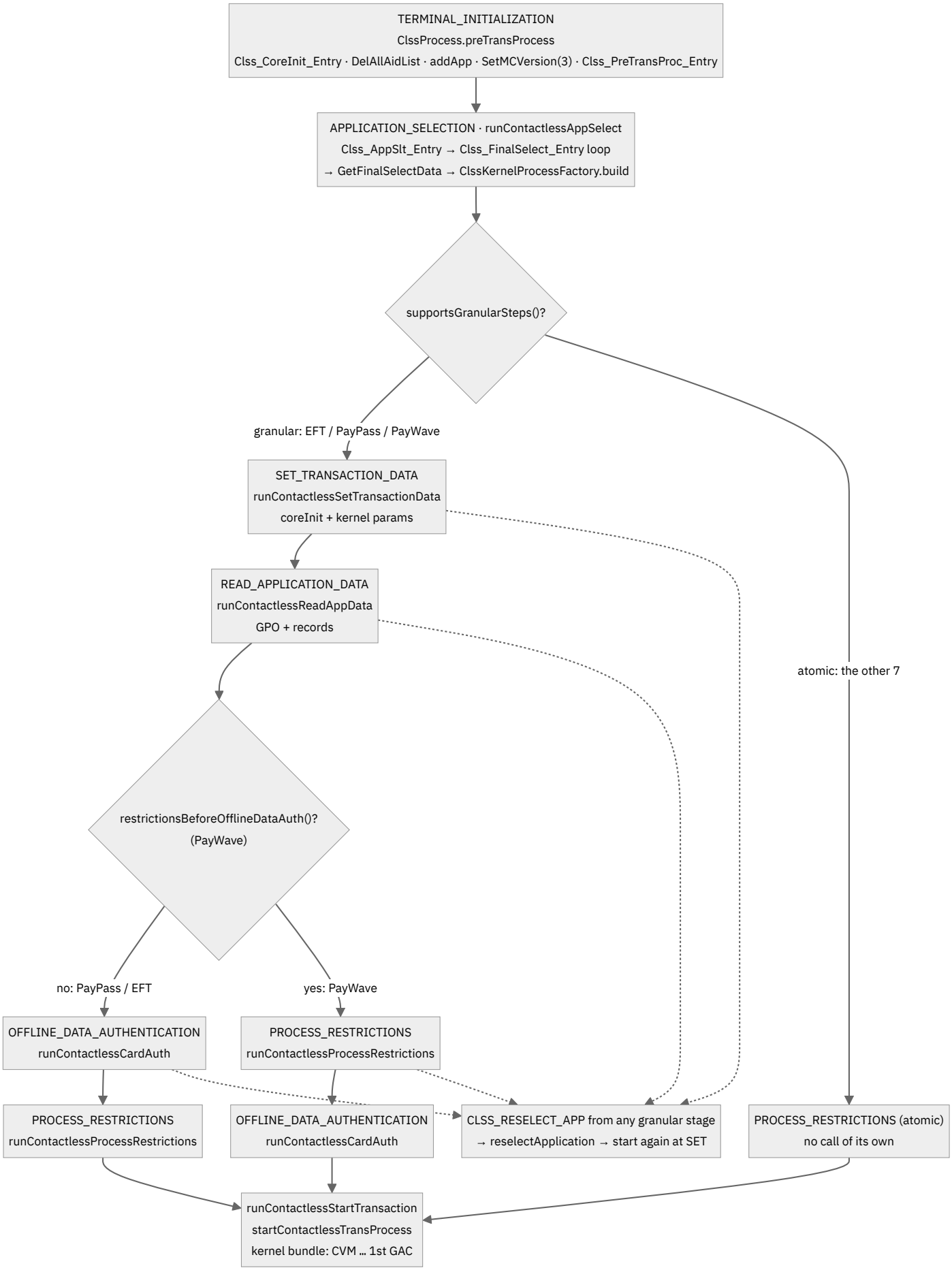
- الـ switch فيه `case ENDING: onEnding()` و `case CLOSING_COMPLETED:` `onClosingCompleted(event)`، ولا `onEnd` ولا `decideEnd`. النتيجة (APPROVED / DECLINED) والنهاية (COMPLETED / ERROR / CANCELLED) تقارير. والـ lock في الأماكن نفسها: اختيار الـ closing التالي وإرساله، ثم تحرير الجهاز وإرسال النهاية.
- لا تداخل: الـ `ReplaySubject` يوصل لكل مشترك بترتيب الـ buffer، ويؤجّل الخطوة للـ observer حتى ينتهي من السابقة. فترى الشاشة مع طلبين: `APPROVED → ENDING → REVERSING → CLOSING_COMPLETED → ACKNOWLEDGING → CLOSING_COMPLETED → COMPLETED`.
- الجهاز مشغول حتى النهاية كما كان. المهلة: حد كل نداء في OkHttp (25 ثانية)، و `closeAtHost` 60 ثانية كشبكة أمان (`1a97dee`).
- ⚠ حد معروف: إذا فشل نداء الـ host أو رمى `RuntimeException` يُرسل `CLOSING_COMPLETED` وتكمل السلسلة. أما `Error` (ليس Exception) فيوقف السلسلة ويبقى الجهاز مشغولًا، وكان الـ loop القديم سيتوقف عنده أيضًا. لم يُختبر على الجهاز.

مكان الفشل (`SaleCommunicationBehavior`)	النتيجة	إلغاء؟
لم يُرسل: CASH_IN، استرداد بلا مرجع، لا EMV result / مفتاح / token، فشل التشفير	`notSent`	لا
`Token refresh: …`	`notSent`	لا
`Exchange: …` أي فشل	`notSent` (لم يُرسل البيع)	لا
`Sale:` / `Refund:` HTTP 401/403 (token / صلاحيات)	`notSent`	لا
`Sale:` / `Refund:` `ConnectException` / `Un knownHostException`	`notSent`	لا
`Sale:` / `Refund:` انتهاء المهلة، انقطاع، 5xx، أي 4xx آخر، envelope تالف	`failed`	نعم
مهلة الـ 90 ثانية في `PosTerminal`	`failed`	نعم

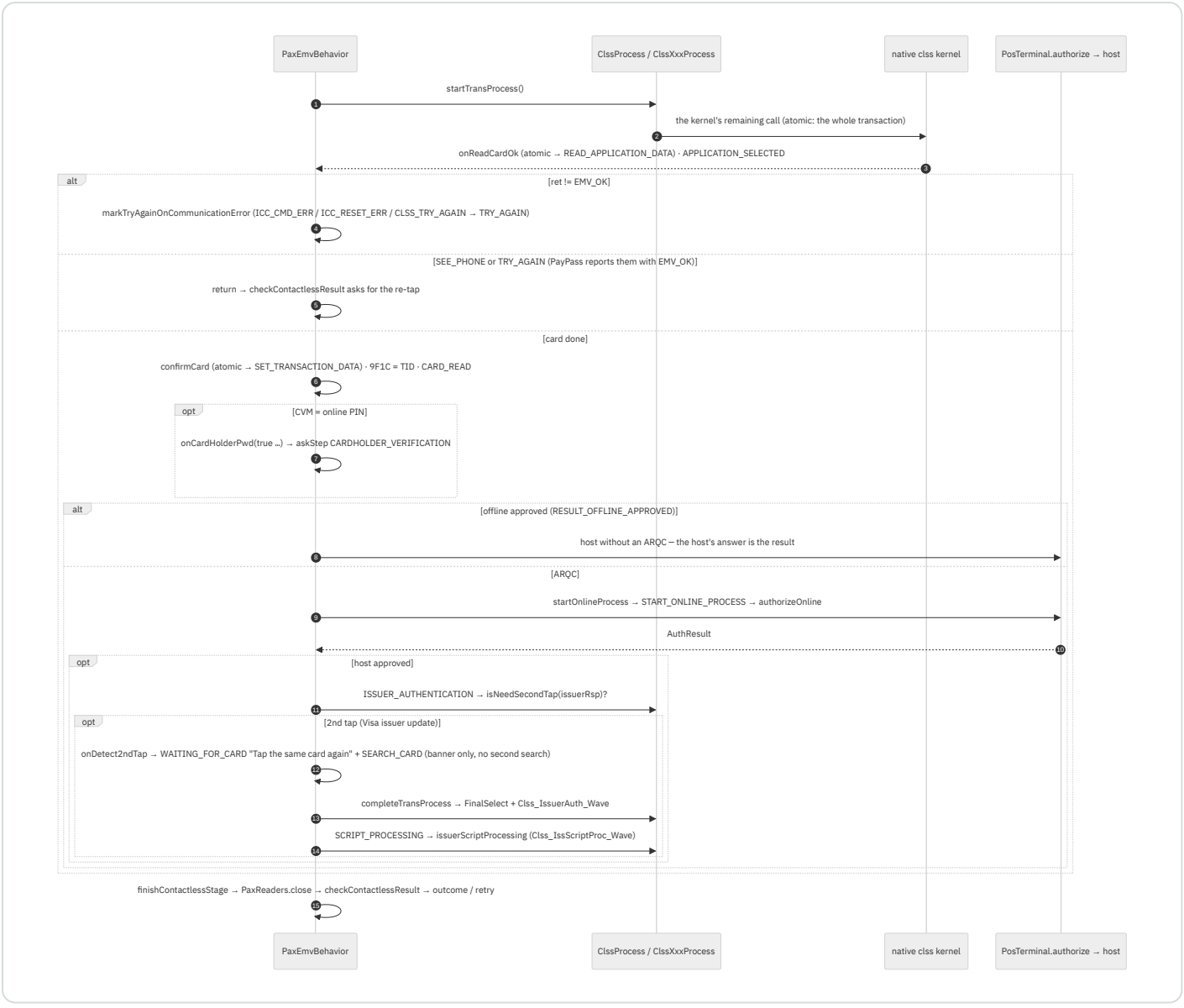
- يظهر اسم المرحلة في رسالة الـ ERROR وفي الـ trace: `Online Failed: Exchange: HTTP 403 Forbidden`.
- الـ trace (tag `HOST`): `ACKNOWLEDGEMENT for request <id>`، `REVERSAL for request <id>`: `approved, not completed`، `REVERSAL …: no usable host answer — Sale: timeout`، `no reversal …: the host could not have approved it — Exchange: HTTP 403 Forbidden`.
- لا يتغير شيء بالنسبة للبطاقة: `failed` و `notSent` كلاهما `RESULT_ONLINE_FAILED` (2nd GENERATE AC بـ AAC). الفرق في الإلغاء فقط.
- ⚠ لم يُختبر على الجهاز ولا مع الـ host. احتياج الإلغاء والتأكيد إلى `mToken` افتراض.

8. مسار الـ contactless

طريقة التشغيل نفسها و `askStep` نفسها، لكن الـ kernel يُختار بعد اختيار التطبيق (`ClssKernelProcessFactory`). هناك نوعان: **granular** (EFT و PayPass و PayWave) ولكل مرحلة فيها استدعاء native مستقل، و**atomic** (Amex و JCB و Mir و PBOC و Pure و RuPay و DPAS) وفيها استدعاء واحد، وتُنشر خطواتها من الـ callbacks. يُسجَّل الـ listener الخاص بالـ kernel مرة واحدة في `runContactlessAppSelect`، ويُحرَّر مرة واحدة في `finishContactlessStage`.


TERMINAL_INITIALIZATION
ClssProcess.preTransProcess
Clss_CoreInit_Entry · DelAllAidList · addApp · SetMCVersion(3) · Clss_PreTransProc_Entry
APPLICATION_SELECTION · runContactlessAppSelect
Clss_AppSlt_Entry → Clss_FinalSelect_Entry loop
→ GetFinalSelectData → ClssKernelProcessFactory.build
supportsGranularSteps()?
granular: EFT / PayPass / PayWave
atomic: the other 7
SET_TRANSACTION_DATA
runContactlessSetTransactionData
coreInit + kernel params
READ_APPLICATION_DATA
runContactlessReadAppData
GPO + records
restrictionsBeforeOfflineDataAuth()?
(PayWave)
no: PayPass / EFT
yes: PayWave
OFFLINE_DATA_AUTHENTICATION
runContactlessCardAuth
PROCESS_RESTRICTIONS
runContactlessProcessRestrictions
PROCESS_RESTRICTIONS
runContactlessProcessRestrictions
OFFLINE_DATA_AUTHENTICATION
runContactlessCardAuth
CLSS_RESELECT_APP from any granular stage
→ reselectApplication → start again at SET
PROCESS_RESTRICTIONS (atomic)
no call of its own
runContactlessStartTransaction
startContactlessTransProcess
kernel bundle: CVM … 1st GAC

بعد المراحل: `startContactlessTransProcess`



EmvStep	PayPass (`ClssPayPassProcess`)	PayWave (`ClssPayWaveProcess`)	EFT (`ClssEFTProcess`)
`SET_TRANSACTION_DATA`	`SetParam_M` ، `Clss_CoreInit_MC` + FinalSelectData + `setPayPass` ،`C Param` (DF8117/18/19/1F، limits)	ReaderParam ، `CoreInit_Wave` `SetVisaAi` + (`aucTmCap` = 9F33) `SetTransData_Wav` ،`dParam_Wave e`	ReaderParam + + `CoreInit_EFT` `SetTransData_EFT` + AidParam
`READ_APPLICATION_DATA`	`Clss_R` + `Clss_InitiateApp_MC` `eadData_MC`	GPO +) `Clss_Proctrans_Wave` (records + AC type	`Clss_Read` + `Clss_InitApp_EFT` `AppData_EFT`
الترتيب بعدها	ODA ثم PR	**PR ثم ODA**	ODA ثم PR
`OFFLINE_DATA_AUTHENTICATION`	تحميل CAPK و revocation فقط (الـ ODA داخل `TransProc`)	`Clss_CardAuth_Wave` (عند TC وليس استردادًا، أو حسب TTQ)	`Clss_Car` + CAPK + `PreTAAProc dAuth_EFT`
`PROCESS_RESTRICTIONS`	لا يوجد استدعاء	`Clss_ProcRestric_Wave`	`Clss_ProcRestrict_EFT`
الـ bundle	`Clss_TransProc_MC_MChip` / `_M` = `ag` ODA+PR+CVM+TRM+TAA+GAC، ثم see phone	`Clss_GetCvmT` : `genTransResult` AC type + `ype_Wave`	`Clss_Post` + `Clss_CVMProc_EFT` `TAAProc_EFT`
2nd tap / completion	`completeTransProcess`	`Clss_IssuerAuth_W` + FinalSelect `ave`	`Clss_CompleteOnlineTrans_EFT` ⚠ لا يُستدعى
`SCRIPT_PROCESSING`	default	`Clss_IssScriptProc_Wave`	default

النتيجة: `checkContactlessResult`

الحالة	الدالة	النهاية
`RESULT_CLSS_SEE_PHONE` (أي ret)	`seePhone` (+ `clsLastNeedSeePhone`)	إعادة المحاولة بالـ CONTACTLESS، وبعد 3 مرات رفض
`RESULT_TRY_AGAIN` (أي ret)	`tryAgain`	إعادة المحاولة بالطريقة نفسها، وبعد 3 مرات رفض
`ONLINE_APPROVED` + OK	`onlineApproved`	APPROVED
`OFFLINE_APPROVED` + OK	`offlineApproved`	"ERROR "Online Failed
OK + غير ذلك	`offlineDenied(CLSS_DECLINE)`	DECLINED
`TRY_ANOTHER_INTERFACE` / `CLSS_USE_CONTAC T`	`tryAnotherInterface`	إعادة المحاولة بالـ CHIP
`ONLINE_DENIED` / `ONLINE_CARD_DENIED` / `OF FLINE_DENIED`	كما في الـ contact	DECLINED
`ONLINE_FAILED`	`onlineFailed`	ERROR

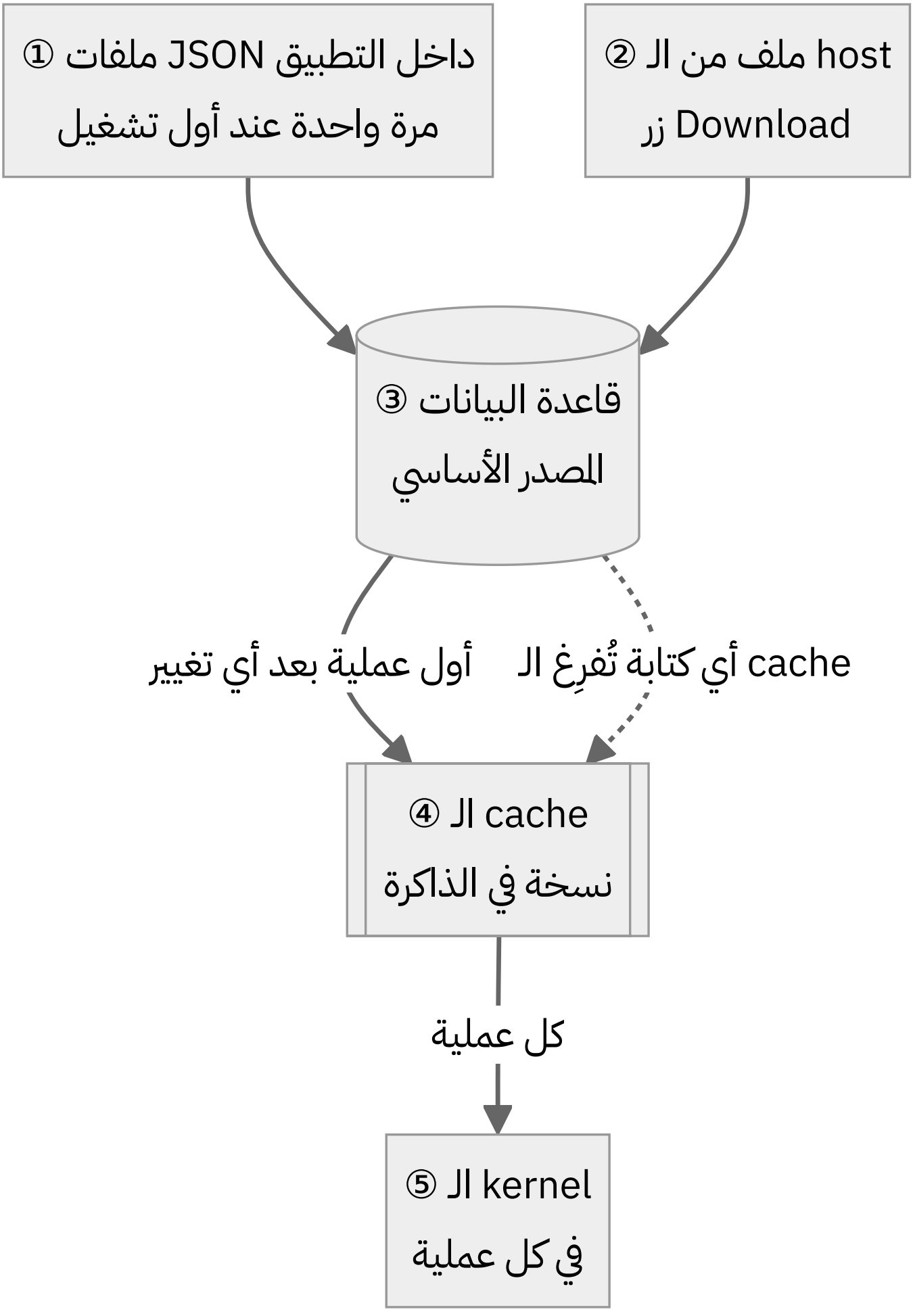
IContactlessCallback	من يستدعيها	ماذا تفعل
onReadCardOk	الـ kernel بعد القراءة	atomic: goToStep(READ_APPLICATION_DATA)؛ APPLICATION_SELECTED
confirmCard	startContactlessTransProcess	atomic: goToStep(SET_TRANSACTION_DATA)؛ 9F1C؛ CARD_READ
onWaitAppSelect / onCardHolderPwd	مشتركة مع الـ contact	askStep
startOnlineProcess	مشتركة	authorizeOnlin → START_ONLINE_PROCESS e
onDetect2ndTap	مرحلة الإكمال	رسالة + SEARCH_CARD (للعرض فقط، دون بحث ثانٍ)
needSeePhone	الـ kernel في الـ tap التالي (Amex / Mir / Pure)	تعيد clsLastNeedSeePhone ثم تصفّرها
onRemoveCard	PayPass / PayWave	تسجيل في الـ log فقط

⚠ **ما زال كذلك:** تنشر `onDetect2ndTap` الرسالة فقط، ولا تنتظر البطاقة قبل `Clss_FinalSelect_Entry` (BACKLOG 1.4). تبقى `clsLastNeedSeePhone` من عملية إلى التي تليها (2.1). تعيد `isNeedSecondTap` القيمة false لـ EFT و Amex و JCB و Pure و PBOC، فلا تُستدعى مرحلة الإكمال أبدًا (2.3). ولا تغلق `finishContactlessStage` أي نافذة.

9. الـ EMV params: مصدرها، ومكان حفظها، ومتى تصل إلى الـ kernel

ما هي الـ params؟ هي القواعد التي يحتاجها الـ kernel ليتخذ قراره في كل عملية: البطاقات المقبولة (الـ AIDs)، والمفاتيح التي يُتحقَّق بها من أن البطاقة أصلية (الـ CAPKs)، والحدود المالية (مثل المبلغ الذي يُطلب بعده PIN في الدفع اللاتلامسي). بدونها لا يستطيع الـ kernel العمل.

الصورة العامة


① ملفات JSON داخل التطبيق
مرة واحدة عند أول تشغيل
② ملف من الـ host
زر Download
③ قاعدة البيانات
المصدر الأساسي
أول عملية بعد أي تغيير
أي كتابة تُفرِغ الـ cache
④ الـ cache
نسخة في الذاكرة
كل عملية
⑤ الـ kernel
في كل عملية

دورة حياة الـ params في 5 خطوات

1. **عند أول تشغيل للتطبيق:** تُملأ قاعدة البيانات من ملفات JSON المرفقة بالتطبيق، حتى يكون هناك ما يعمل به. يحدث ذلك مرة واحدة فقط.
2. **عند الضغط على Download:** يرسل الـ host ملفًا (XML للـ contact وآخر للـ contactless)، فيُقرأ ويُكتب في قاعدة البيانات بدلًا من القديم.
3. **قاعدة البيانات هي المصدر الأساسي:** القيم الفعلية للـ params موجودة فيها.
4. **الـ cache نسخة سريعة:** أول عملية بعد أي تغيير تقرأ قاعدة البيانات كاملة مرة واحدة وتحتفظ بها في الذاكرة، والعمليات التالية تأخذ من الذاكرة مباشرة.
5. **الـ kernel لا يحتفظ بشيء:** في **كل** عملية تُحمَّل إليه الـ params من الـ cache من جديد، وتُمسح منه بانتهاء العملية.

مثال

غيّر الـ host حدّ طلب الـ PIN في الدفع اللاتلامسي من 600 إلى 1000 جنيه:

- **Download:** أصبحت القيمة في قاعدة البيانات 1000، وأُفرِغ الـ cache.
- **أول عملية بعدها:** يُعاد ملء الـ cache من قاعدة البيانات (1000)، ويستلمها الـ kernel.
- **عملية tap بمبلغ 800 جنيه:** تتم دون طلب PIN ✓

ماذا تعرض شاشة EMV Params؟

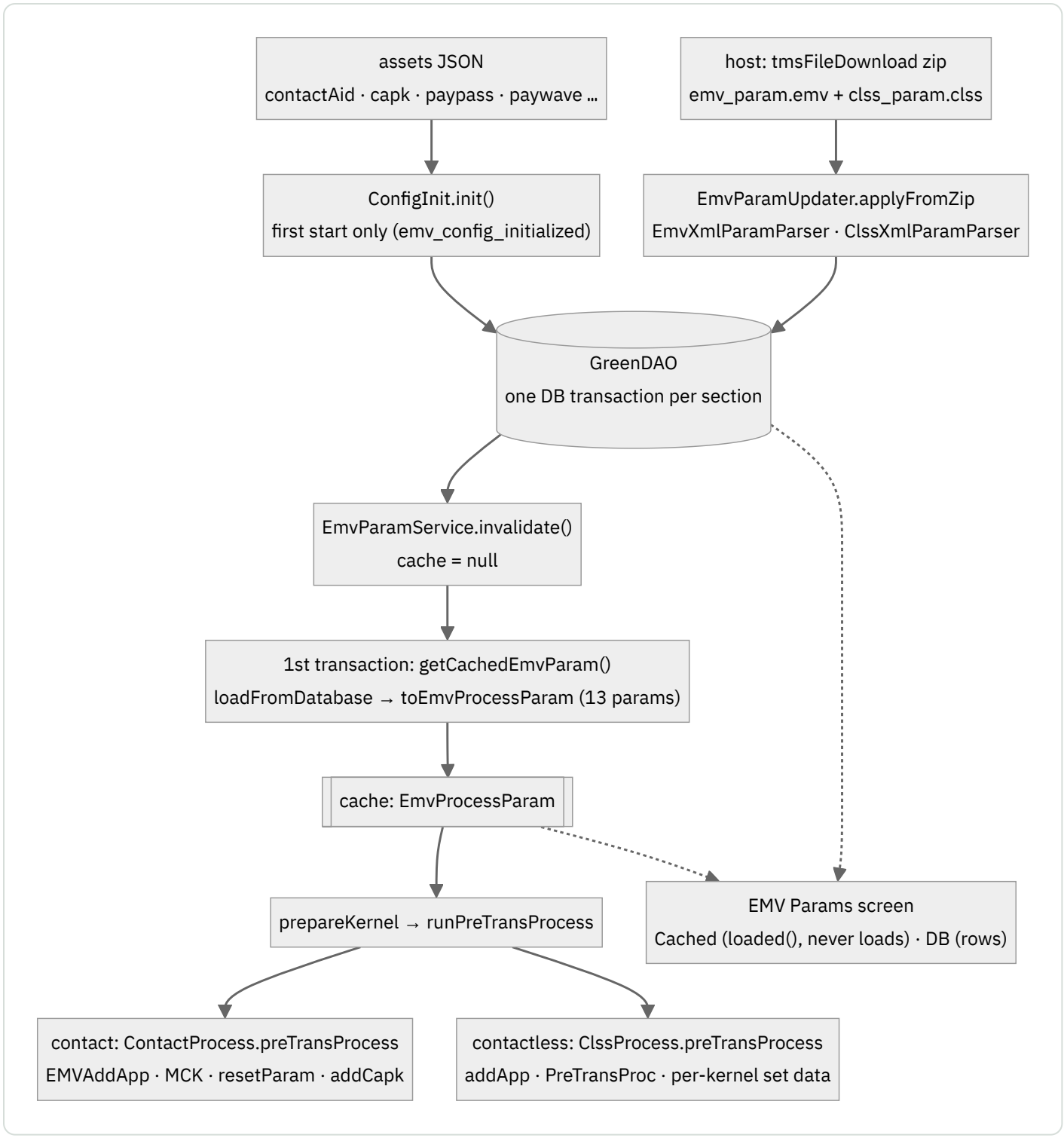
- **DB:** القيم المحفوظة (المصدر الأساسي).
- **Cached:** القيم التي سيستلمها الـ kernel الآن. إن كانت فارغة فمعنى ذلك أنه لم تتم أي عملية منذ آخر تغيير، وليس أن هناك خطأ.

قواعد مهمة

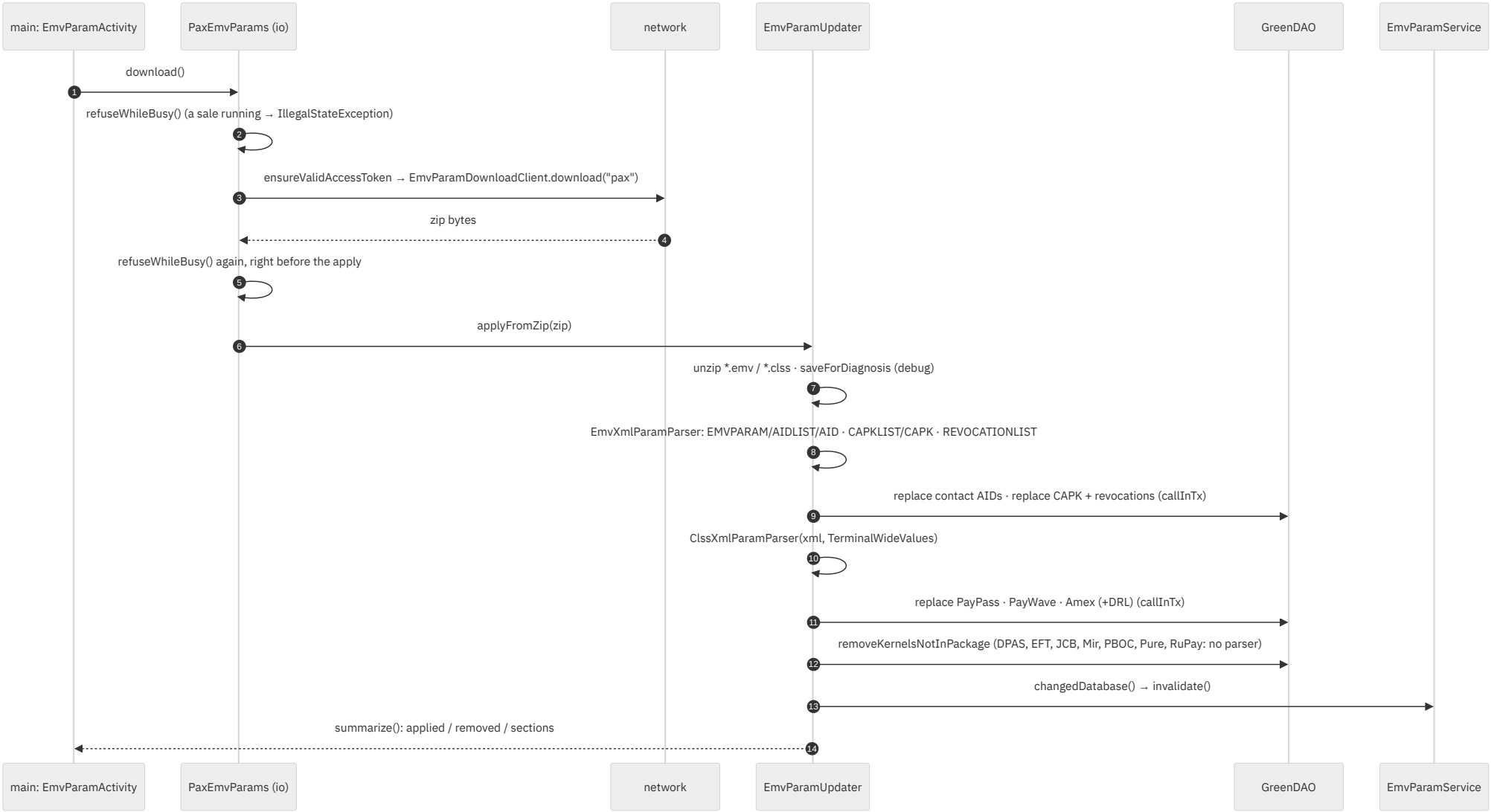
- **الـ contact:** إذا لم يتضمن ملف الـ host جزءًا معينًا، تبقى القيم القديمة كما هي. **الـ contactless:** ملف الـ host هو المرجع الكامل، وأي نوع بطاقة غير موجود فيه يُحذف.
- بيانات الجهاز (MCC، MID، TID، اسم التاجر) **لا** تأتي من ملف الـ host.
- ⚠ لا يوجد فحص لصلاحية الـ CAPKs: 76 من 88 في ملفات الـ JSON منتهية الصلاحية.

التفاصيل التقنية (للمراجعة)

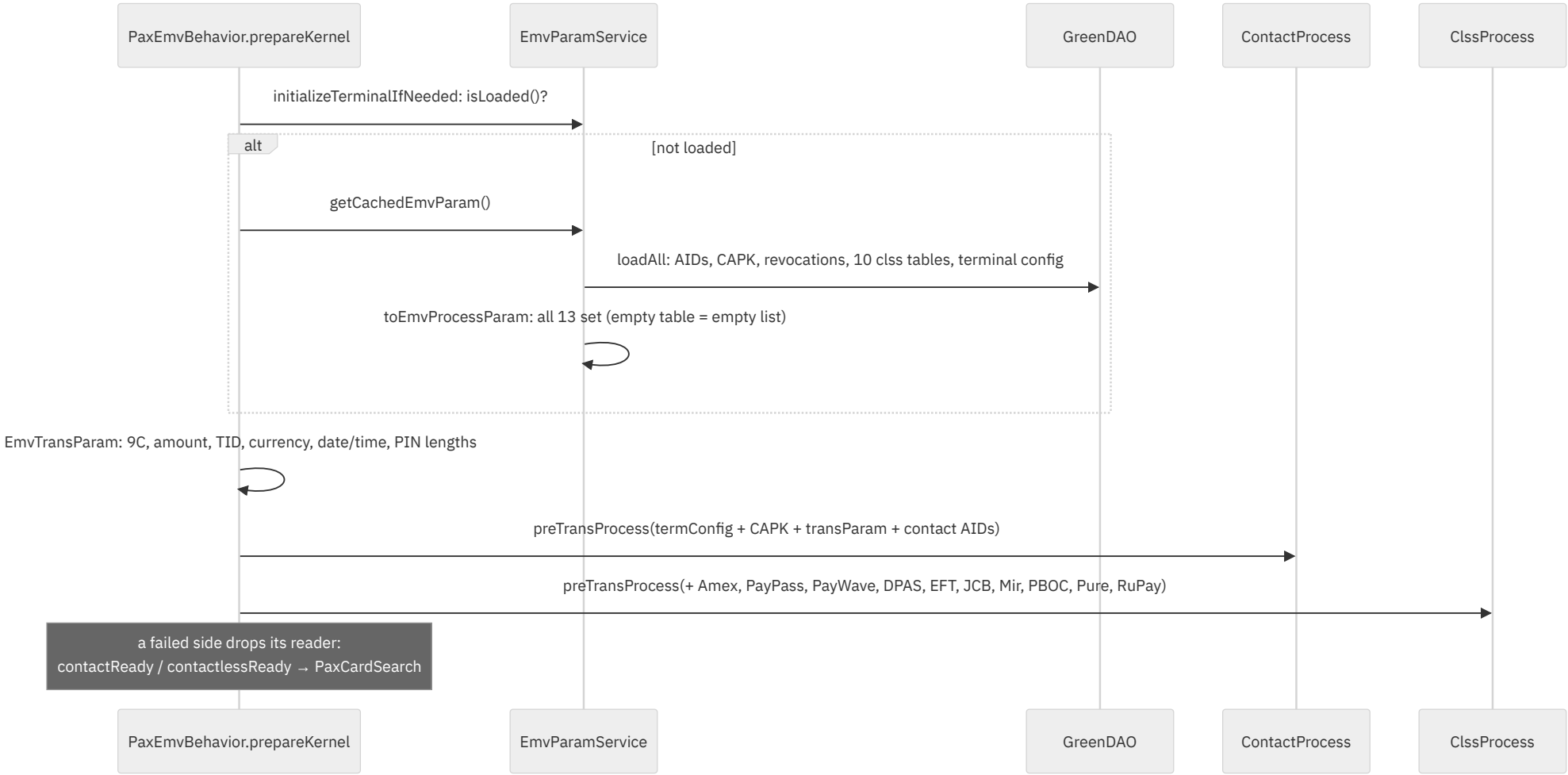
المسار بأسماء الكلاسات: يكتب `ConfigInit` (أول تشغيل) و `EmvParamUpdater.applyFromZip` (التنزيل) في GreenDAO؛ وتُفرغ `EmvParamService.invalidate()` الـ cache؛ `getCachedEmvParam()` → `loadFromDatabase` → `toEmvProcessParam` (param 13)؛ `prepareKernel` → `preTransProcess` للـ contact والـ contactless.



التنزيل خطوة بخطوة


main: EmvParamActivity
PaxEmvParams (io)
network
EmvParamUpdater
GreenDAO
EmvParamService
download()
1
refuseWhileBusy() (a sale running → IllegalStateException)
2
ensureValidAccessToken → EmvParamDownloadClient.download("pax")
3
zip bytes
4
refuseWhileBusy() again, right before the apply
5
applyFromZip(zip)
6
unzip *.emv / *.clss · saveForDiagnosis (debug)
7
EmvXmlParamParser: EMVPARAM/AIDLIST/AID · CAPKLIST/CAPK · REVOCATIONLIST
8
replace contact AIDs · replace CAPK + revocations (callInTx)
9
ClssXmlParamParser(xml, TerminalWideValues)
10
replace PayPass · PayWave · Amex (+DRL) (callInTx)
11
removeKernelsNotInPackage (DPAS, EFT, JCB, Mir, PBOC, Pure, RuPay: no parser)
12
changedDatabase() → invalidate()
13
summarize(): applied / removed / sections
14
main: EmvParamActivity
PaxEmvParams (io)
network
EmvParamUpdater
GreenDAO
EmvParamService

أول عملية بعد التنزيل

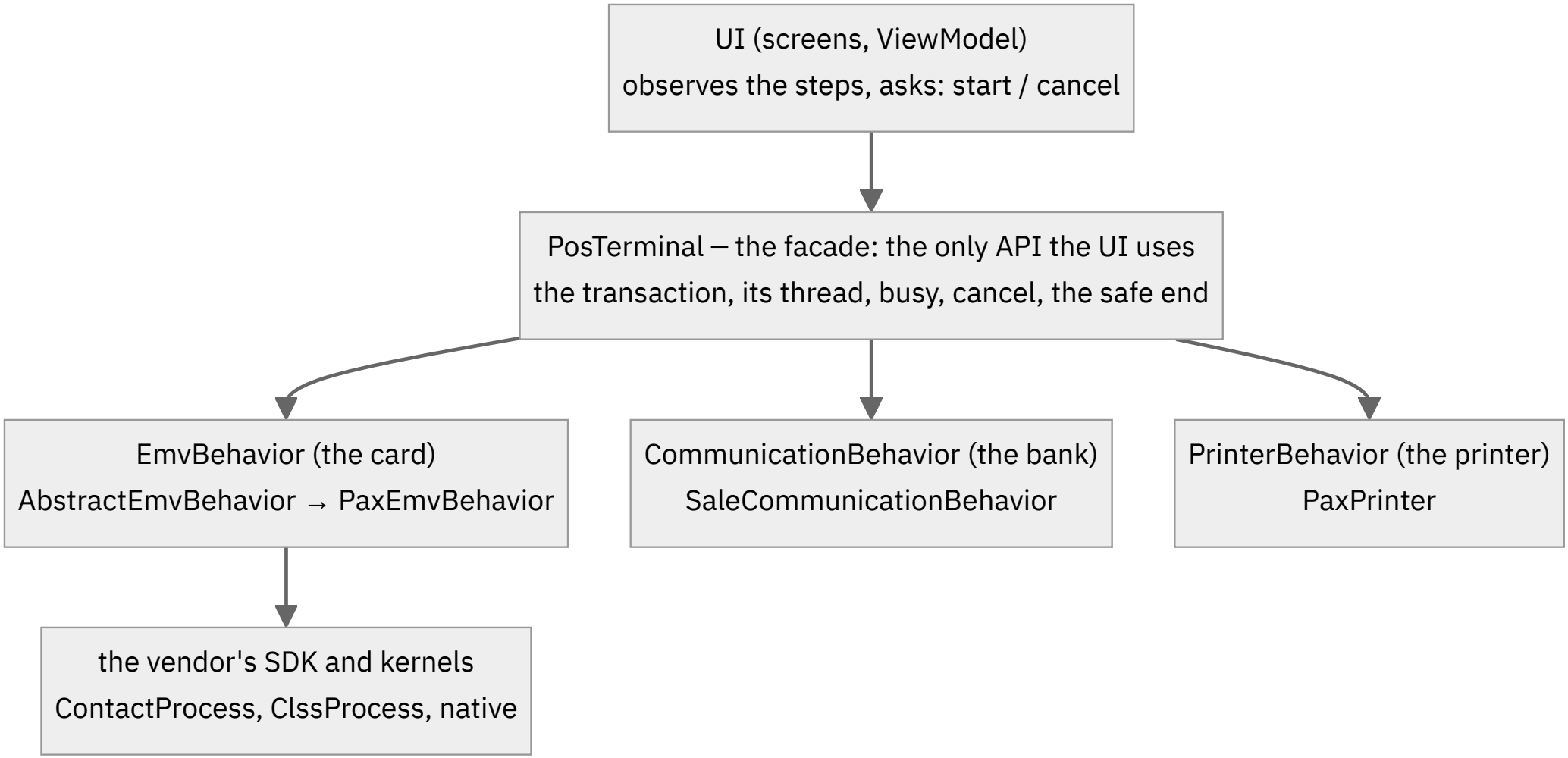

PaxEmvBehavior.prepareKernel
EmvParamService
GreenDAO
ContactProcess
ClssProcess
initializeTerminalIfNeeded: isLoaded()?
alt
[not loaded]
getCachedEmvParam()
loadAll: AIDs, CAPK, revocations, 10 clss tables, terminal config
toEmvProcessParam: all 13 set (empty table = empty list)
EmvTransParam: 9C, amount, TID, currency, date/time, PIN lengths
preTransProcess(termConfig + CAPK + transParam + contact AIDs)
preTransProcess(+ Amex, PayPass, PayWave, DPAS, EFT, JCB, Mir, PBOC, Pure, RuPay)
a failed side drops its reader:
contactReady / contactlessReady → PaxCardSearch
PaxEmvBehavior.prepareKernel
EmvParamService
GreenDAO
ContactProcess
ClssProcess

الـ param	أين يدخل الـ kernel
الـ contact AIDs، وأول AID كقيمة افتراضية	`ContactProcess.preTransProcess` (`EMVAddApp`، `setEmvAndMCKPara`m)
9F33 / 9F40 / 9F35 الخاصة بالـ AID المختار	`emvSetParam` → `resetParam` (داخل `EMVReadAppData`)
CAPK و revocation	contact: `cardAuthentication` → `addCapk`؛ contactless: مرحلة الـ ODA
الـ contactless AIDs والحدود	`ClssEntryAddAid.addApp` + `Clss_PreTransProc_Entry`
TTQ، 9F33، floor، CVM limit، DF81xx	`setTransactionData` لكل (granular) kernel أو داخل الاستدعاء الواحد (atomic)
الـ TID في field 55 للـ contactless	`setTlv(9F1C)` → `confirmCard`

- `OTHERCONFIGURATION` (الـ TID و الـ MCC وبيانات التاجر) لا يُؤخذ من الـ XML، وهذا قرار مقصود.
- يُتجاهل أي CAPK بلا RID أو KeyID صحيح. ⚠ لا يوجد فحص للصلاحية ولا للـ checksum.
- أعاد `5755adb` الدوال الـ 12 `setXxxParam` الناقصة في `loadFromDatabase`، وهو إصلاح الـ crash في `preTransProcess`، ويحميه `EmvParamServiceCacheTest`.

10. مبادئ وأنماط البرمجة في المشروع (ببساطة)

كل مبدأ في سطر: معناه ببساطة، وأين يظهر في الكود. لا شيء هنا للزينة؛ كل واحد يحل مشكلة حقيقية في الدفع.


UI (screens, ViewModel)
observes the steps, asks: start / cancel
PosTerminal — the facade: the only API the UI uses
the transaction, its thread, busy, cancel, the safe end
EmvBehavior (the card)
AbstractEmvBehavior → PaxEmvBehavior
CommunicationBehavior (the bank)
SaleCommunicationBehavior
PrinterBehavior (the printer)
PaxPrinter
the vendor's SDK and kernels
ContactProcess, ClssProcess, native

المبدأ / النمط	ببساطة	أين في الكود
Layers + Facade	طبقات، كل طبقة تكلم التي تحتها فقط. والـ UI يرى بابًا واحدًا يخفي ما خلفه	`PosTerminal` هو الباب الوحيد للـ UI (`startTran` / `cancel` / `steps()` / `acceptCard` / `saction()`). ليس طبقة تمرير فارغة: يملك العملية فعلًا (القاعدة 5 تمنع الطبقات التي تمرّر فقط)
Strategy / Polymorphism	واجهة واحدة وأكثر من تنفيذ، يُختار واحد منها	`EmvBehavior` (PAX / Ingenico / Fake)، `Commu`nicationBehavior، `PrinterBehavior`
Open / Closed	نضيف بدل أن نعدّل	vendor جديد = flavor جديد بـ `TerminalFactory` وكلاساته، والـ core لا يتغيّر
Template Method	الكلاس الأب يحدد الترتيب، والابن يملأ الخطوات	`AbstractEmvBehavior` : `run` و `goToStep` والـ dispatch، و 17 `onXxx` يملؤها الـ vendor؛ `author`izeOnline() `final`
Observer	من يهمه الأمر يشترك، ويصله كل جديد	`Transaction.steps()` (`ReplaySubject`) والشاشة مشتركة؛ `EmvEngine.emvSteps` و `EmvSt`epObserver؛ `TransactionStepObserver`
Callback	تعطي جهة "رقمًا تتصل بك عليه"	`Authorizer` (5§)، `CardSearchListener`، `IC`ontactCallback و `askStep` (4§)
Adapter	محوّل بين شكلين لا يعرف أحدهما الآخر	`PaxHostAnswer` (`AuthResult` → شكل PAX)؛ `ContactProcess.EmvCallBackListener` (native → listener)؛ `PaxEmvBehavior` بصفته `I`ContactCallback
Factory / Builder	مكان واحد يقرر كيف يُنشأ الشيء	`TerminalFactory` (لكل flavor)، `ClssKernelPr`ocessFactory (لكل kernel) لكل scheme)، `AuthResu`lt.approved / declined / failed / notSen`t`، `TransactionStepEvent.builder(...)`
Dependency Injection (يدوي)	الكلاس يأخذ ما يحتاجه من الخارج ولا يُنشئه	`PaxTerminal` → `super(behavior, communic`ation, printer)؛ و `attach(this::authoriz`e) للدائرة الوحيدة
Dependency Inversion (الـ D في SOLID)	الطبقة العليا تعتمد على واجهة، لا على كلاس محدد	`AbstractEmvBehavior` يعرف `Authorizer`، لا `PosTerminal` ؛ الـ core يعرف `CommunicationBeh`avior، لا Retrofit
Single Responsibility	كل كلاس له شغلة واحدة	`PaxCardSearch` (البحث)، `PaxPinEntry` (الـ PIN)، `PaxHostAnswer` ، `PaxReaders` ، `PaxCar`dData، `PaxSetup` ، `PaxKernelSession`
Single source of truth	كل حالة لها مالك واحد ونسخة واحدة	الجهاز مشغول = `PosTerminal.current` ؛ حالة العملية كلها = `Transaction.end` ؛ خطواتها = `Tran`saction.steps
Immutability / Value Object	الشيء لا يتغيّر بعد إنشائه؛ التعديل يعطي نسخة جديدة	`AuthResult` ، `TransactionStepEvent` ، `Tran`sactionConfig.withXxx(...) يرجع config جديدًا
Fail closed	عند الشك نرفض، لا نوافق	لا رد = online failed؛ لا PIN = `USER_CANCEL` ؛ الـ callback يرد دائمًا (`setCallBackResult`)